0_10xMouse_PBMC_SoloTE

August 31, 2026

[1]:
```
library(Seurat)
library(ggplot2)
library(pheatmap)
library(dplyr)
library(RColorBrewer)
library(clustree)
library(reshape2)
getwd()
dir.create("figures_10xMouse_PBMC")
dir.create("data_10xMouse_PBMC")
dataset_id <- "10xMouse_PBMC"
sample_id <- "10k"
colorPBMC <- "#81B29A"
```

```
Loading required package: SeuratObject

Loading required package: sp

'SeuratObject' was built under R 4.3.3 but the current version is
4.4.1; it is recomended that you reinstall 'SeuratObject' as the ABI
for R may have changed

'SeuratObject' was built with package 'Matrix' 1.6.4 but the current
version is 1.7.5; it is recomended that you reinstall 'SeuratObject' as
the ABI for 'Matrix' may have changed


Attaching package: 'SeuratObject'


The following objects are masked from 'package:base':

    intersect, t


Warning message:
"package 'dplyr' was built under R version 4.4.3"
```

```
Attaching package: ‘dplyr’


The following objects are masked from ‘package:stats’:

    filter, lag


The following objects are masked from ‘package:base’:

    intersect, setdiff, setequal, union


Warning message:
"package ‘RColorBrewer’ was built under R version 4.4.3"
Loading required package: ggraph


Attaching package: ‘ggraph’


The following object is masked from ‘package:sp’:

    geometry


Warning message:
"package ‘reshape2’ was built under R version 4.4.3"
```

’/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets’

```
Warning message in dir.create("figures_10xMouse_PBMC"):
"'figures_10xMouse_PBMC' already exists"
Warning message in dir.create("data_10xMouse_PBMC"):
"'data_10xMouse_PBMC' already exists"
```

[3]:
```
getwd()

conversionTable <- read.table("annotation/annotation_mm10_conversion_withAge.
 ↪tsv") # created with annotation_scripts/create_annotations_mouse.Rmd
head(conversionTable)
```

’/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets’

A data.frame: 6 × 16

	chr <chr>	start <int>	end <int>	strand <chr>	sw_score <int>	locusID <chr>	family <chr>	subfam <chr>
1	chr1	3000001	3002128	-	12955	L1_Mus3_dup24	L1	L1-Mu
2	chr1	3003153	3003994	-	1216	L1Md_F_dup2	L1	L1Md-
3	chr1	3003994	3004054	-	234	L1_Mus3_dup25	L1	L1-Mu
4	chr1	3004041	3004206	+	3685	L1_Rod_dup1	L1	L1-Roc
5	chr1	3004271	3005001	+	3685	L1_Rod_dup2	L1	L1-Roc
6	chr1	3005002	3005439	+	1280	L1_Rod_dup3	L1	L1-Roc

1 Import SoloTE results and split genes and TEs

[144]:
```
SoloTE_path <- paste0("/mnt/volume_1p5T/results/SoloTEout/", dataset_id, "/",␣
 ↪sample_id, "/", sample_id,"_SoloTE_output/", sample_id,"_legacytes_MATRIX")
STAR_path <- paste0("/mnt/TEresults/snakemake_results/results/STARoutdir/
 ↪",dataset_id,"/",sample_id,"/best_Solo.out/Gene")
filteredBarcodes <- read.table(paste0(STAR_path, "/filtered/barcodes.tsv"))$V1␣
 ↪# read barcodes seleced by STARsolo
```

[145]:
```
# remove cells annotated as multiplets

# import cell assignment done by 10x
mouse_assignment <- read.csv("data_10xMouse_PBMC/
 ↪SC3_v3_NextGem_DI_CellPlex_Mouse_PBMC_10K_Multiplex_multiplexing_analysis_assignment_confid
 ↪csv")
mouse_assignment$Barcodes <- sapply(strsplit(mouse_assignment$Barcodes,␣
 ↪split="-"), "[", 1)

multiplets <-␣
 ↪mouse_assignment$Barcodes[mouse_assignment$Assignment=="Multiplet"]


filteredBarcodes <- setdiff(filteredBarcodes, multiplets)
```

[149]:
```
legacyTEmatrix <- Seurat::ReadMtx(mtx = paste0(SoloTE_path, "/matrix.mtx"),
                                  cells = paste0(SoloTE_path, "/barcodes.tsv"),
                                  features = paste0(SoloTE_path, "/features.tsv"))␣
 ↪# read matrix
legacyTEmatrix <- legacyTEmatrix[,filteredBarcodes]

# select TEs
TEs <- grep("SoloTE", rownames(legacyTEmatrix), value = T)
locusTEs <- grep("chr", TEs, value = T)
# subset the matrix keeping only TEs
TEmatrix <- legacyTEmatrix[locusTEs,]
# remove "SoloTE" from the name of the TEs
rownames(TEmatrix) <- gsub("SoloTE\\|", "", rownames(TEmatrix))
```

```
rownames(TEmatrix) <- gsub("\\|", "-", rownames(TEmatrix))
rownames(TEmatrix) <- gsub("\\_", "-", rownames(TEmatrix))
rownames(TEmatrix) <- gsub("\\?", "", rownames(TEmatrix))

#table(rownames(TEmatrix) %in% conversionTable$soloteID)
# transform into Stellarscope IDs
#rownames(TEmatrix) <- conversionTable$stellarscopeID[match(rownames(TEmatrix),
 ↪conversionTable$soloteID)]
nCells <- ncol(TEmatrix)
thrMinCells <- round(nCells * 0.05)

# create Seurat object with shallow filtering of TEs expressed in at least 5%
 ↪of cells and cells expressing at least 50 TEs
objTE <- Seurat::CreateSeuratObject(TEmatrix, project = "PBMC",
                                    min.cells = thrMinCells, min.features = 20)
objTE
```

```
An object of class Seurat
5430 features across 4787 samples within 1 assay
Active assay: RNA (5430 features, 0 variable features)
 1 layer present: counts
```

[150]:
```
# select genes
genes <- setdiff(rownames(legacyTEmatrix), TEs)
# subset the matrix keeping only genes
Genematrix <- legacyTEmatrix[genes,]
# create Seurat object with shallow filtering of genes expressed in at least 50
 ↪cells and cells expressing at least 100 genes
Gene <- Seurat::CreateSeuratObject(Genematrix, project = "PBMC",
                           min.cells = thrMinCells, min.features = 200)

#filteredBarcodes <- read.table(paste0(STAR_path, "/filtered/barcodes.tsv"))$V1
 ↪# read barcodes seleced by STARsolo


objTE_SoloTE <- objTE[,filteredBarcodes] # keep only filtered barcodes
objGenes_SoloTE <- Gene[,filteredBarcodes] # keep only filtered barcodes

objTE_SoloTE
objGenes_SoloTE
```

```
An object of class Seurat
5430 features across 4787 samples within 1 assay
Active assay: RNA (5430 features, 0 variable features)
 1 layer present: counts
```

```
An object of class Seurat
```

```
8652 features across 4787 samples within 1 assay
Active assay: RNA (8652 features, 0 variable features)
 1 layer present: counts
```

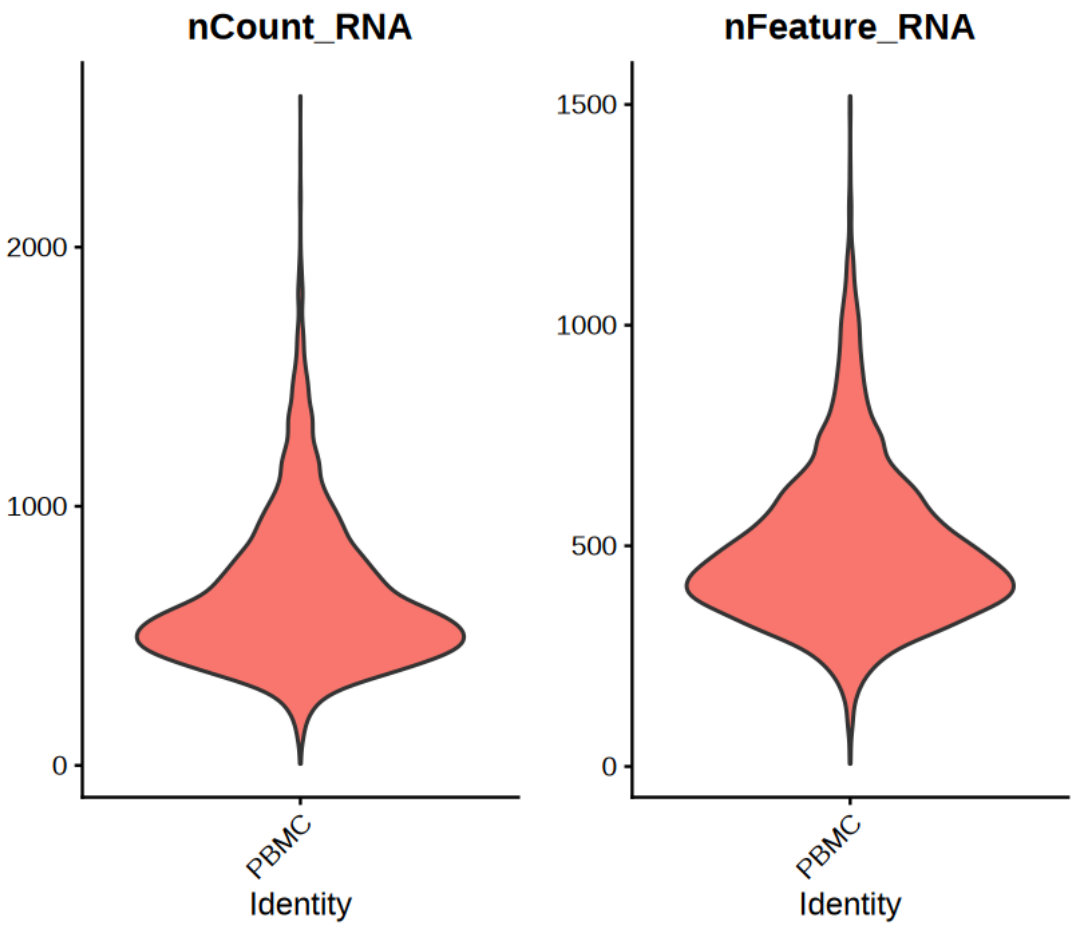
2 QC

[159]:
```r
# QC TEs

options(repr.plot.width=7, repr.plot.height=6)

objTE_SoloTE@meta.data$nCount_TE <- objTE_SoloTE@meta.data$nCount_RNA
objTE_SoloTE@meta.data$nFeature_TE <- objTE_SoloTE@meta.data$nFeature_RNA

# # Visualize QC metrics as a violin plot
VlnPlot(objTE_SoloTE, features = c("nCount_RNA", "nFeature_RNA"), ncol = 2,
        pt.size = 0, alpha = 0.5)
summary(objTE_SoloTE$nCount_RNA)
summary(objTE_SoloTE$nFeature_RNA)
```

```
Warning message:
"Default search for "data" layer in "RNA" assay yielded no results; utilizing
"counts" layer instead."
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    6.0   458.0   577.0   654.8   783.0  2583.0
```

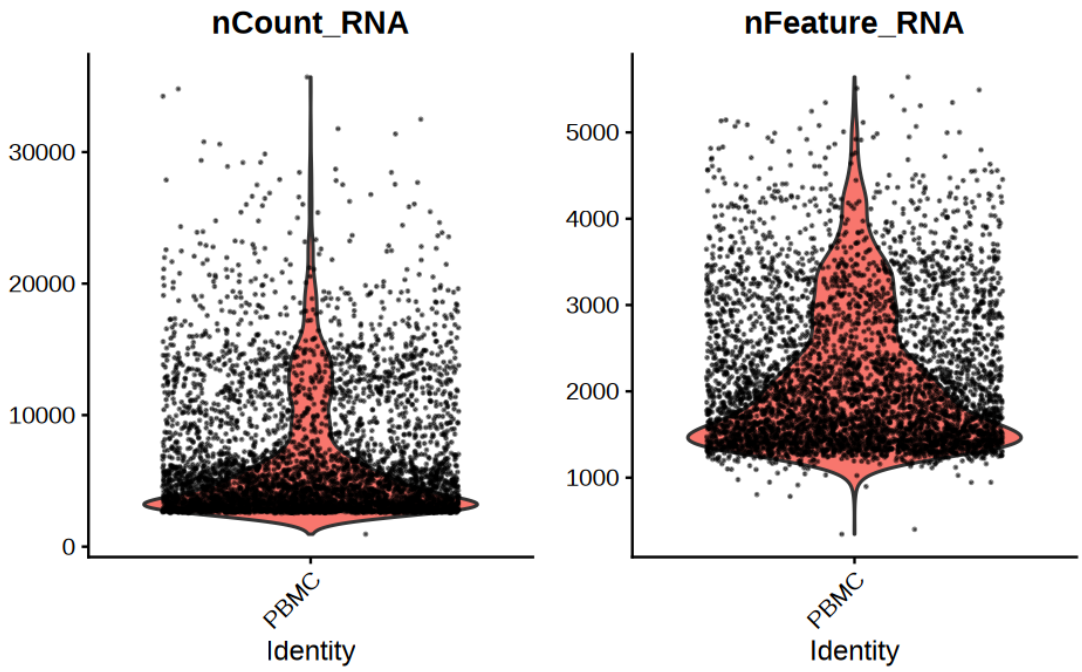
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    6.0   376.0   459.0   496.5   582.0  1519.0
```



[163]:
```
# QC genes
options(repr.plot.width=8, repr.plot.height=5)

VlnPlot(objGenes_SoloTE, features = c( "nCount_RNA", "nFeature_RNA" ), ncol =␣
↪2,
        pt.size = 0.05, alpha = 0.5, group.by = "orig.ident")

summary(objGenes_SoloTE$nFeature_RNA)
summary(objGenes_SoloTE$nCount_RNA)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    345    1500    1909    2169    2666    5641
```

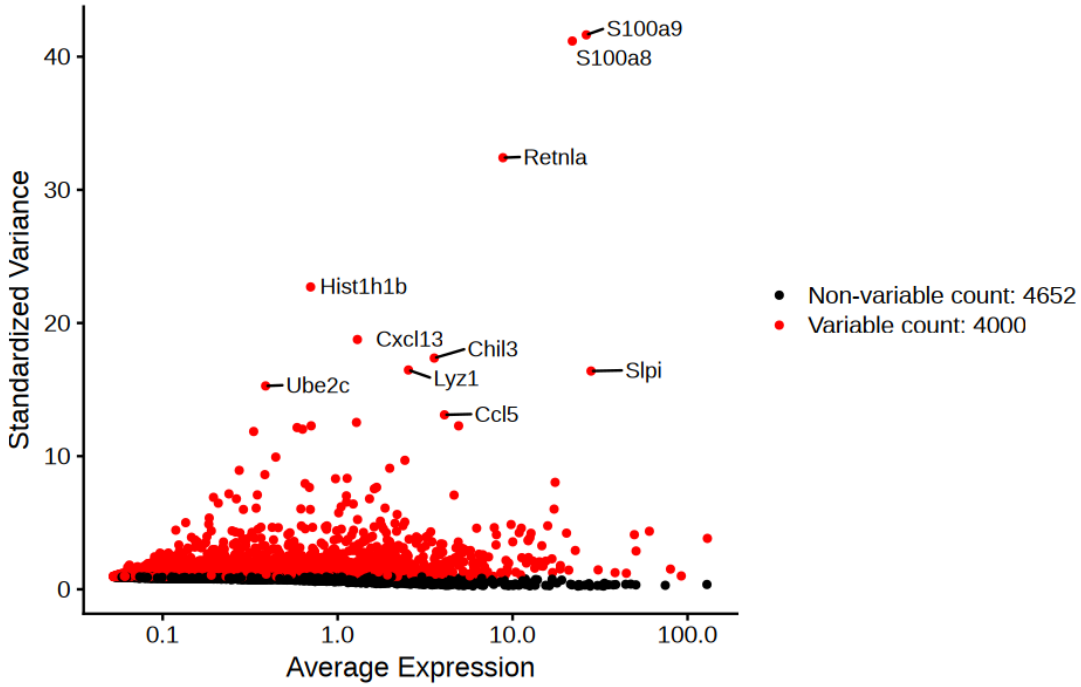
```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    949    3205    4644    6688    8544   35702
```



3 Genes

[164]:
```
objGenes_SoloTE <- NormalizeData(objGenes_SoloTE, normalization.method =␣
 ↪"LogNormalize", scale.factor = 10000)

objGenes_SoloTE <- FindVariableFeatures(objGenes_SoloTE, selection.method =␣
 ↪"vst", nfeatures = 4000)

# Identify the 10 most highly variable genes
top10 <- head(VariableFeatures(objGenes_SoloTE), 10)

# plot variable features with and without labels
plot1 <- VariableFeaturePlot(objGenes_SoloTE)
plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
plot2
```

```
Normalizing layer: counts

Finding variable features for layer counts

When using repel, set xnudge and ynudge to 0 for optimal results
```



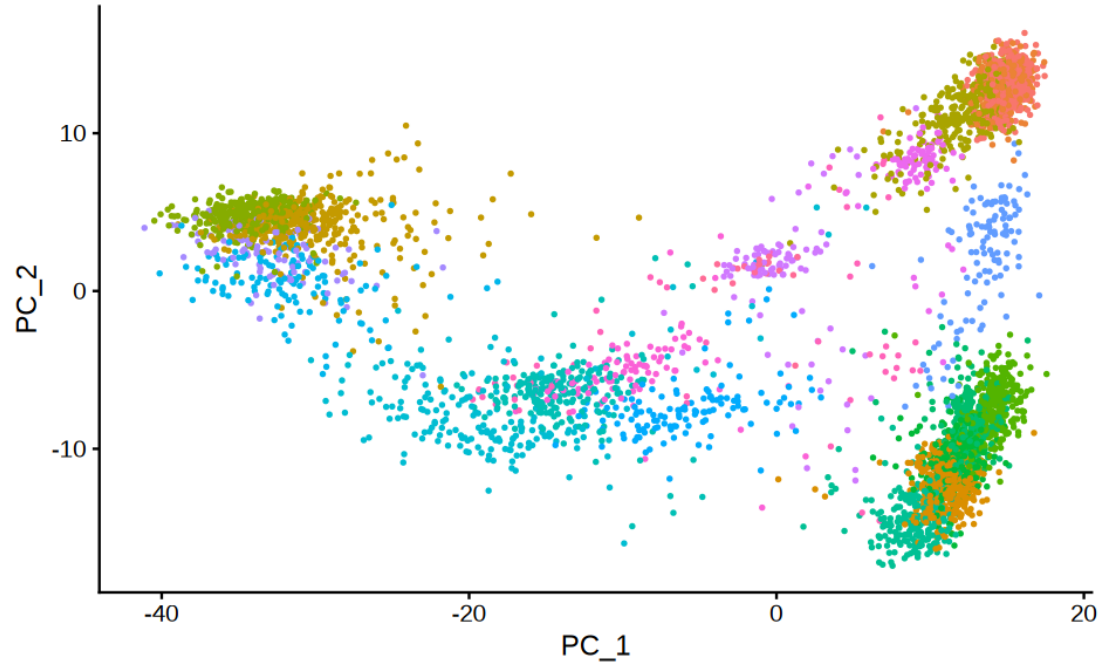
[165]:
```
gc()
all.genes <- rownames(objGenes_SoloTE)
objGenes_SoloTE <- ScaleData(objGenes_SoloTE) # on hugs

objGenes_SoloTE <- RunPCA(objGenes_SoloTE, features = VariableFeatures(object =␣
↪objGenes_SoloTE))


DimPlot(objGenes_SoloTE, reduction = "pca") + NoLegend()

ElbowPlot(objGenes_SoloTE)
```

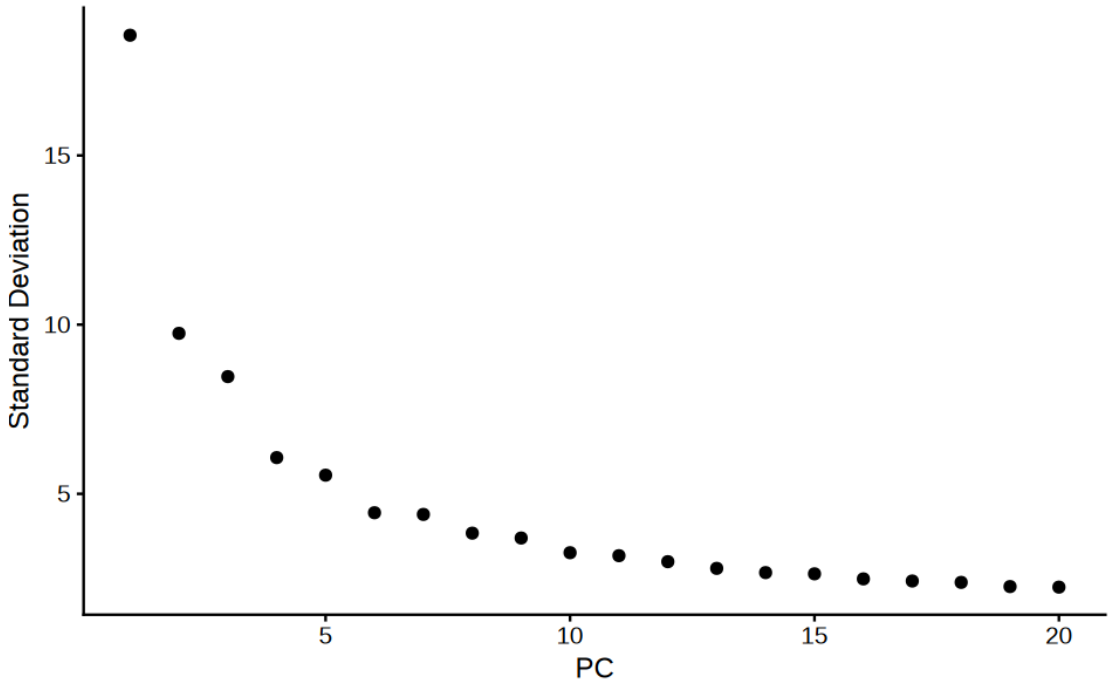
A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	23994959	1281.5	60565990	3234.6	60565990	3234.6
Vcells	362560980	2766.2	742181096	5662.4	742131242	5662.1

```
Centering and scaling data matrix

PC_ 1
Positive:  Ltb, Cd69, Cd2, Gm8369, Ccr7, Rhoh, Gimap3, Satb1, Vps37b, Smad7
           Ighm, Gimap4, Bcl2, Gimap5, Gramd3, Trbc2, Cd3d, Cd3g, Ms4a4b,
C230085N15Rik
           Lef1, Ly6d, Cd79a, Il7r, Cd79b, Ebf1, Itk, Skap1, Igkc, Tcf7
Negative:  Emilin2, App, Itgam, Lyz2, Cd14, Fn1, Alox5ap, Wfdc17, Gda, Fcgr3
           Cxcl2, Cfp, Ccrl2, Trf, Ifitm2, Grn, Ccl6, Sdc3, Ifitm3, Lrp1
```

```
           Thbs1, C1qb, Ccl9, Ecm1, Adgre1, Ltc4s, C1qc, Csf1r, Plek, Mt1
PC_ 2
Positive:  Cd3d, Trbc2, Ms4a4b, Cd3g, Skap1, Itk, Il7r, Lef1, Txk, Cd3e
           Lat, Tcf7, Vps37b, Trac, Rgs10, Bcl11b, Lck, Klk8, Satb1, Thy1
           Ms4a6b, Trbc1, Cd247, Smad7, Igfbp4, Prkcq, Cd8b1, Prdx6,
C230085N15Rik, Nkg7
Negative:  Cd74, Napsa, H2-Eb1, H2-Aa, H2-Ab1, Cd79b, Ly6d, Cd79a, Ms4a1, Mzb1
           Pkig, Plac8, Bank1, H2-DMa, Iglc2, H2-DMb2, Ighm, Siglecg, Igkc, Ebf1
           Mef2c, S100a6, Tcf4, Iglc3, Ly86, Pou2f2, Ctsh, Pou2af1, Rassf4,
Ifi30
PC_ 3
Positive:  Plbd1, Ccr2, Fam129a, Lst1, Gpr141, Clec4a3, Ltb4r1, Clec4a1,
Cd300c2, Ms4a6c
           Il1b, Cx3cr1, Olfm1, Tnip3, Clec4a2, Clec7a, Clec12a, Adrbk2,
Ccdc88a, Marcks
           Lgals3, Plxnd1, Clec4b1, Ptpro, Hpgd, Sowahc, Ifitm6, Il13ra1, Aif1,
Gm2a
Negative:  Alox15, Ltbp1, Selp, Icam2, Prg4, Cd5l, Serpinb2, Ptgis, Ednrb,
Serpinb1a
           Itga6, C4b, Flnb, Garnl3, Tgfb2, Pf4, Cd24a, Pmp22, Cd79a, F5
           Cd79b, Ly6d, Bank1, Ms4a1, Mzb1, Iglc2, Timd4, C1qc, Cd81, Igkc
PC_ 4
Positive:  Stmn1, Mki67, Cks1b, Cdca8, Hist1h1b, Top2a, Mcm5, Ube2c, Lig1, Tyms
           Smc2, Gins2, Ptma, Tuba1b, Tubb5, Mcm3, Mcm7, Tacc3, Gmnn, Ran
           Dut, Hist1h2ap, Rrm1, Pola1, Dctpp1, Ezh2, Hells, Mcm6, Ranbp1,
Incenp
Negative:  Csf3r, Hdc, S100a9, Hp, Lst1, S100a8, Pla2g7, Mgst1, Slc16a3, Gsr
           Pbx1, Apoe, Msrb1, Trem3, Pglyrp1, Lrrk2, Slfn4, 6430548M08Rik,
Zfyve9, Trem1
           C5ar1, Adgre4, Sorl1, Ets2, Plaur, Cebpb, Grina, Tmcc1, Tgfb2, Clec4e
PC_ 5
Positive:  Mrc1, Tnip3, Slamf9, Clec4b1, H2-DMb1, Cysltr1, Batf3, Olfm1,
Tmem176a, Pltp
           Ppp1r15a, Retnla, Mgl2, Dapk1, Nr4a3, Cd209a, H2-Ab1, H2-Eb1, H2-Aa,
Clec10a
           Trerf1, Tmem176b, Mt2, Sdc4, Mir155hg, Tnfsf9, Avpi1, Gm14636, Ckb,
Cd74
Negative:  I830127L07Rik, Hp, Chil3, S100a8, Trem3, S100a9, Gsr, Stmn1,
Hist1h1b, C3
           Cdca8, Pglyrp1, Ube2c, Lst1, Msrb1, Mki67, Top2a, Hmgb2, Hist1h2ap,
Lmnb1
           Sgms2, Clec12a, Mgst1, Megf9, Ly6c2, Gpr141, Adpgk, Cebpd, Kif23,
Cd63
```





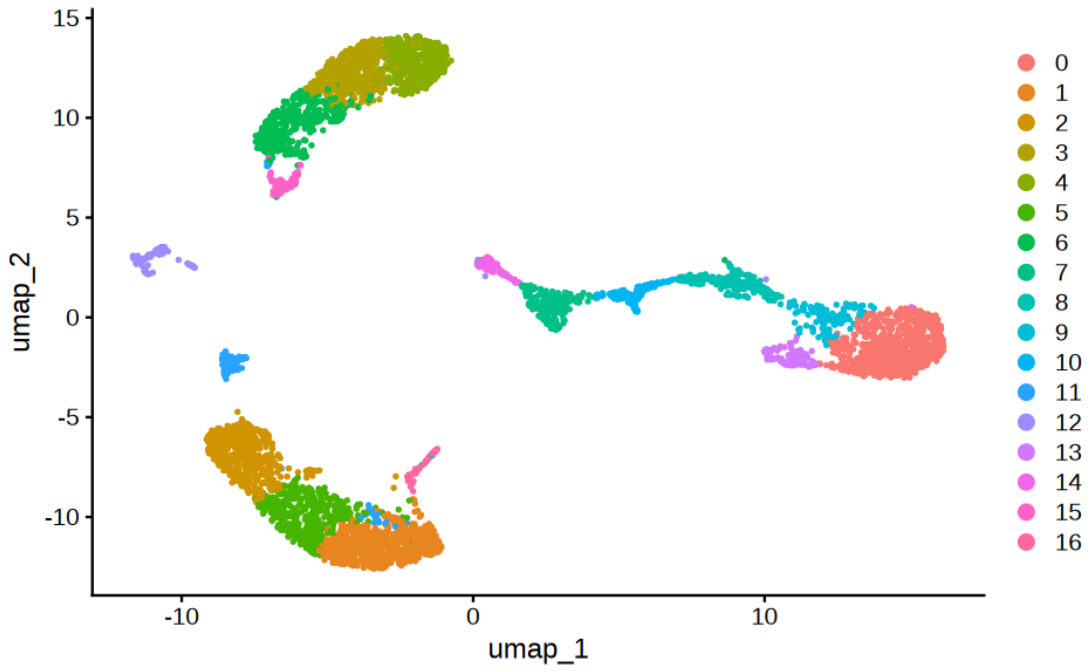
[166]:
```
objGenes_SoloTE <- FindNeighbors(objGenes_SoloTE, dims = 1:10, k.param = 20)
objGenes_SoloTE <- FindClusters(objGenes_SoloTE, resolution = 1)
```

```
objGenes_SoloTE <- RunUMAP(objGenes_SoloTE, dims = 1:10)
DimPlot(objGenes_SoloTE, reduction = "umap")
```

```
Computing nearest neighbor graph

Computing SNN


Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8618
Number of communities: 17
Elapsed time: 0 seconds
10:57:32 UMAP embedding parameters a = 0.9922 b = 1.112

10:57:32 Read 4787 rows and found 10 numeric columns

10:57:32 Using Annoy for neighbor search, n_neighbors = 30

10:57:32 Building Annoy index with metric = cosine, n_trees = 50

0%   10   20   30   40   50   60   70   80   90   100%

[----|----|----|----|----|----|----|----|----|----|

*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
```

```
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
|

10:57:33 Writing NN index file to temp file /tmp/RtmpNvYOrB/file1b13191141e79d

10:57:33 Searching Annoy index using 1 thread, search_k = 3000

10:57:34 Annoy recall = 100%

10:57:38 Commencing smooth kNN distance calibration using 1 thread
 with target n_neighbors = 30

10:57:46 Initializing from normalized Laplacian + noise (using RSpectra)

10:57:46 Commencing optimization for 500 epochs, with 192946 positive edges

10:57:46 Using rng type: pcg
```

```
10:57:56 Optimization finished
```



4 Celltype annotation

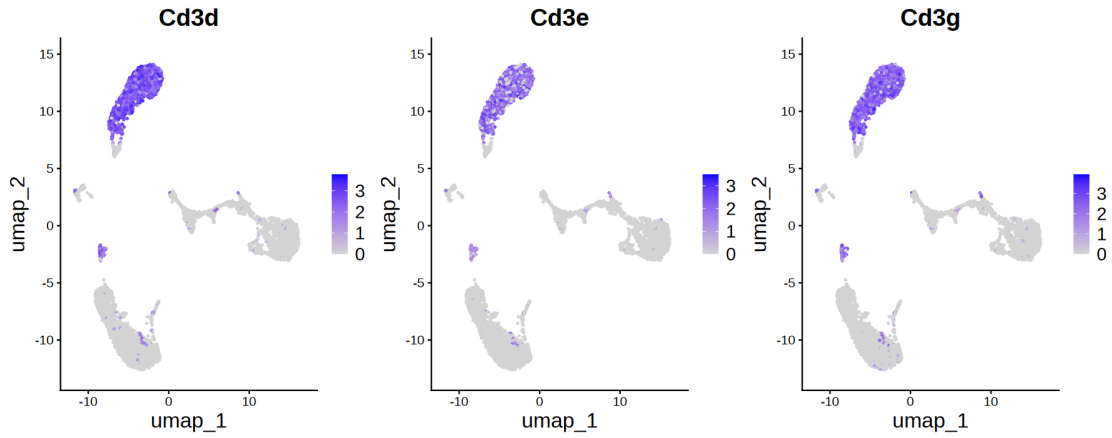
4.1 Plot markers

[167]:
```
mouse_pbmc_markers_broad <- list(
  "T_cells"       = c("Cd3d", "Cd3e", "Cd3g"),
  "NK_cells"      = c("Nkg7"),
  "B_cells"       = c("Cd79a", "Cd79b", "Ms4a1"),
  "Monocytes"     = c("Lyz2", "Csf1r", "Ccr2", "Fcgr4"),
  "DCs"           = c("Flt3", "H2-Ab1"),
  "Platelets"     = c("Ppbp", "Pf4"),
  "Neutrophils"   = c("S100a8", "S100a9", "Ly6g", "Mpo")
)
```

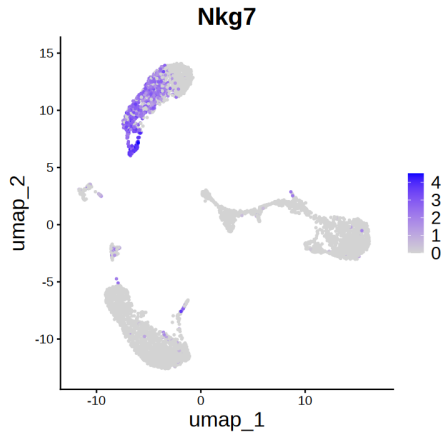
[168]:
```
options(repr.plot.width=15, repr.plot.height=6)

for(celltype in names(mouse_pbmc_markers_broad)){
    print(mouse_pbmc_markers_broad[celltype])
    show(FeaturePlot(objGenes_SoloTE, reduction = "umap",
        features=mouse_pbmc_markers_broad[[celltype]], ncol=3) &
```

```
        theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))␣
  ↪)
}
```

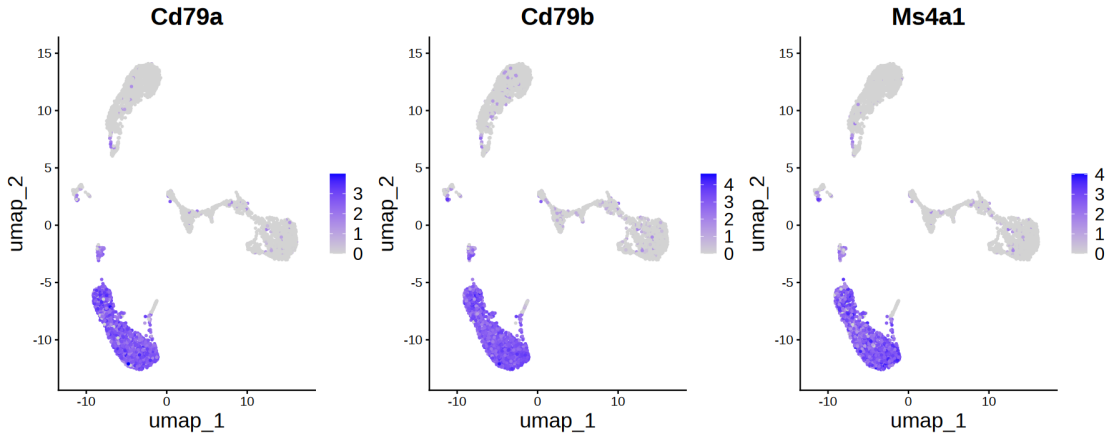
```
$T_cells
[1] "Cd3d" "Cd3e" "Cd3g"

$NK_cells
[1] "Nkg7"
```



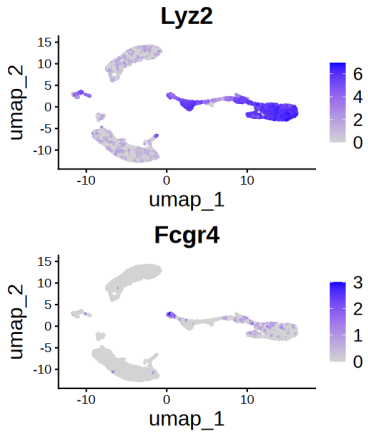

```
$B_cells
[1] "Cd79a" "Cd79b" "Ms4a1"
```



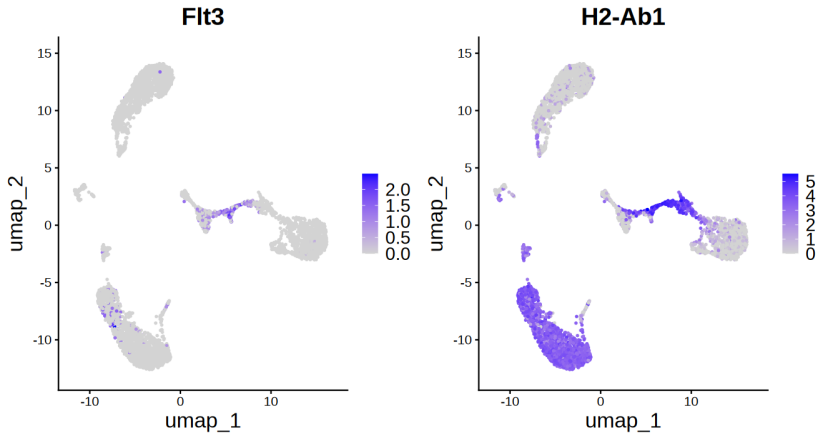

```
$Monocytes
[1] "Lyz2"  "Csf1r" "Ccr2"  "Fcgr4"
```




```
$DCs
[1] "Flt3"   "H2-Ab1"
```

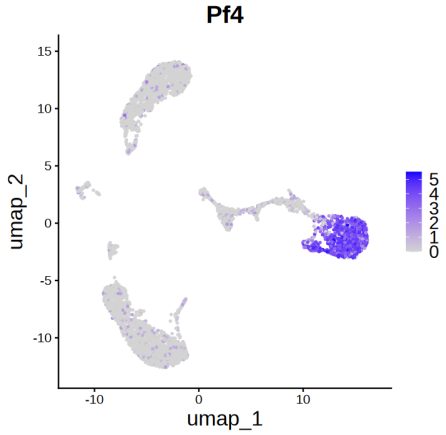


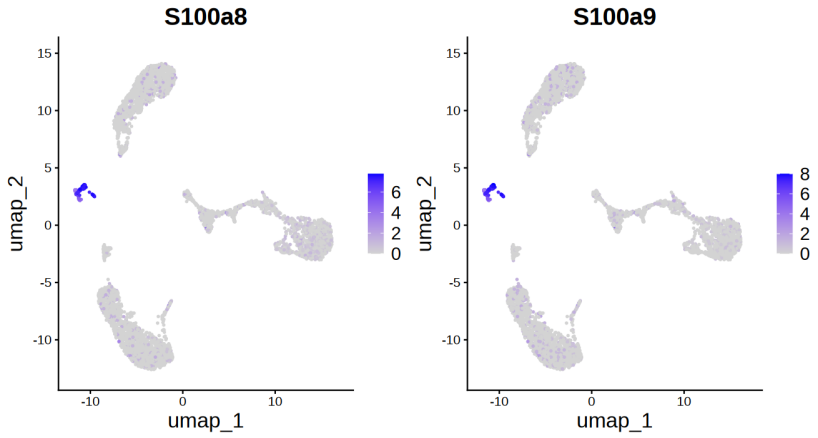

```
$Platelets
[1] "Ppbp" "Pf4"

Warning message:
"The following requested variables were not found: Ppbp"
```



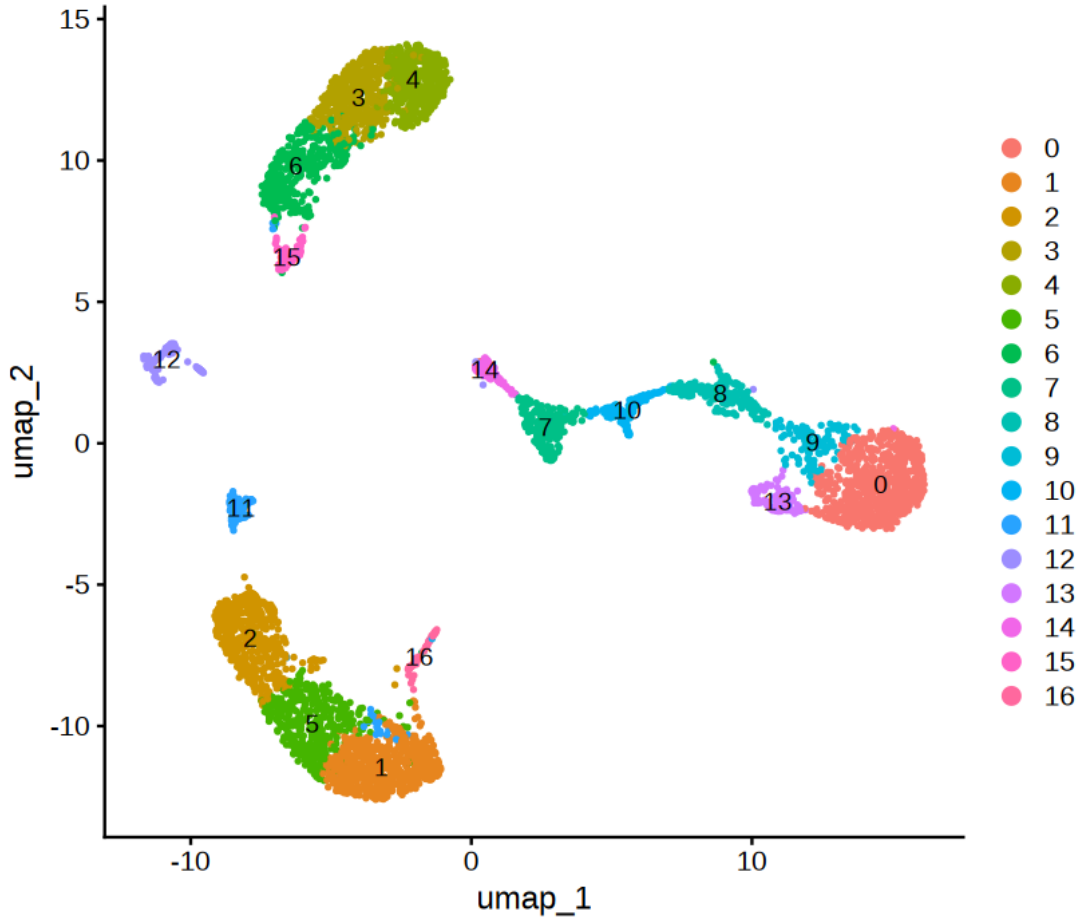

```
$Neutrophils
[1] "S100a8" "S100a9" "Ly6g"   "Mpo"
```

```
Warning message:
"The following requested variables were not found: Ly6g, Mpo"
```





[170]:
```
options(repr.plot.width=7, repr.plot.height=6)

DimPlot(objGenes_SoloTE, reduction = "umap", label=TRUE)
```

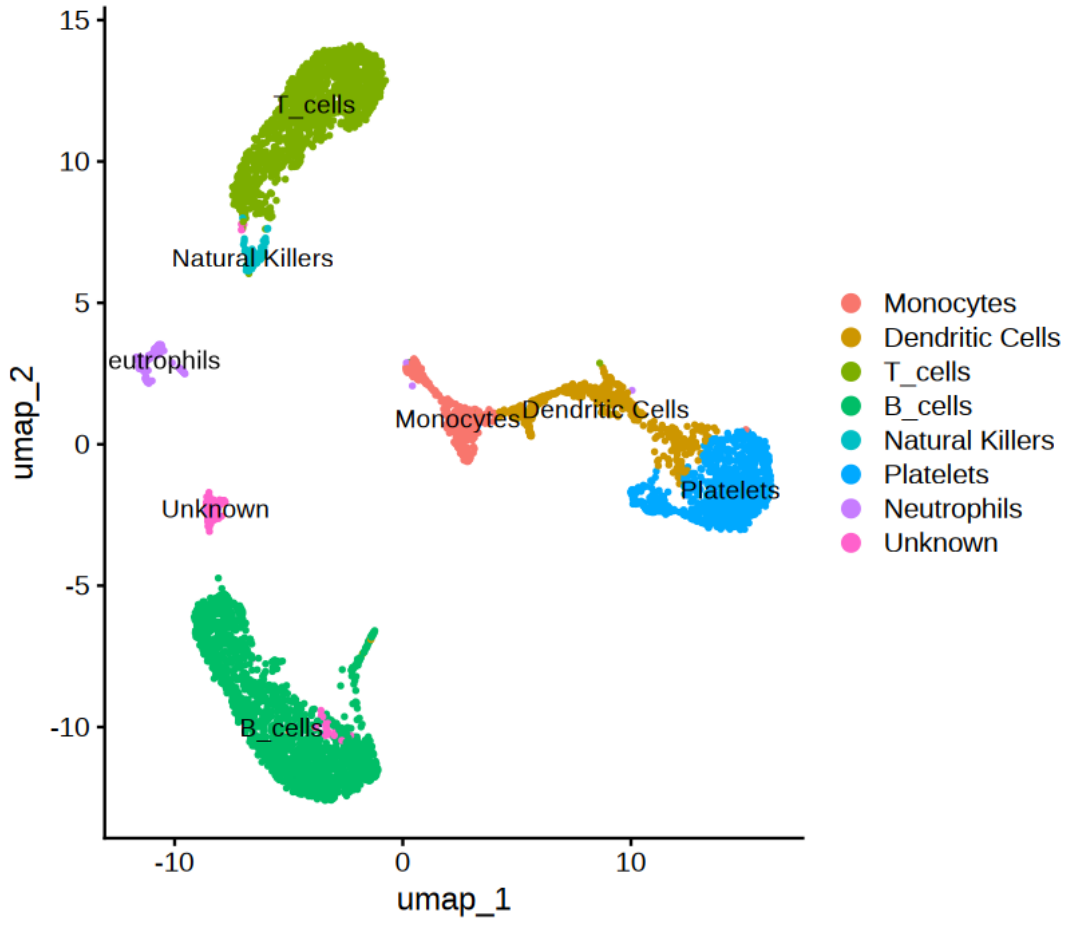


[171]:
```
# 1. Define the mapping from cluster number -> cell type
#    Names are cluster IDs (as they appear in Idents(seu) or␣
 ↪seu$seurat_clusters),
#    values are the cell type labels.
cluster_to_celltype <- c(
  "0" = "Platelets",
  "1" = "B_cells",
  "2" = "B_cells",
  "3" = "T_cells",#"T_CD8",
  "4" = "T_cells",#"T_CD4",
  "5" = "B_cells",
  "6" = "T_cells",#"T_CD8",
  "7" = "Monocytes",
  "8" = "Dendritic Cells",
  "9" = "Dendritic Cells",
  "10" = "Dendritic Cells",
  "11" = "Unknown",
  "12" = "Neutrophils",
  "13" = "Platelets",
  "14" = "Monocytes",
  "15" = "Natural Killers",
  "16" = "B_cells"
 )

# 2. Make sure the cluster identities are set as the active idents
#    (adjust "seurat_clusters" if your cluster column has a different name)
Idents(objGenes_SoloTE) <- "seurat_clusters"

# 3. Map cluster labels to cell types and store as a new metadata column
objGenes_SoloTE$celltype <- as.vector(cluster_to_celltype[as.
 ↪character(Idents(objGenes_SoloTE))])

# 4. (Optional) set celltype as the active identity for downstream plotting
Idents(objGenes_SoloTE) <- "celltype"

# 5. Sanity check
table(objGenes_SoloTE$celltype, objGenes_SoloTE$seurat_clusters)
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
B_cells	0	626	498	0	0	426	0	0	0	0	0	0	0	0	0
Dendritic Cells	0	0	0	0	0	0	0	0	188	125	125	0	0	0	0
Monocytes	0	0	0	0	0	0	0	216	0	0	0	0	0	0	91
Natural Killers	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Neutrophils	0	0	0	0	0	0	0	0	0	0	0	0	113	0	0
Platelets	716	0	0	0	0	0	0	0	0	0	0	0	0	112	0

```
T_cells            0   0   0 467 447   0 380   0   0   0   0   0   0   0   0
Unknown            0   0   0   0   0   0   0   0   0   0   0 116   0   0   0

                  15  16
B_cells            0  51
Dendritic Cells    0   0
Monocytes          0   0
Natural Killers   90   0
Neutrophils        0   0
Platelets          0   0
T_cells            0   0
Unknown            0   0
```

[172]:
```
options(repr.plot.width=7, repr.plot.height=6)

DimPlot(objGenes_SoloTE, reduction = "umap", label=TRUE)
```



[173]:
```r
# Replace with your actual ambiguous cluster IDs
Idents(objGenes_SoloTE) <- "seurat_clusters"

# Get top markers for each ambiguous cluster vs. all others
markers <- FindAllMarkers(
  objGenes_SoloTE,
  only.pos = TRUE,                # upregulated genes only, easier to interpret
  min.pct = 0.25,                 # gene must be in 25% of cells in the cluster
  logfc.threshold = 0.25
)

head(markers)
```

```
Calculating cluster 0


Calculating cluster 1

Calculating cluster 2

Calculating cluster 3

Calculating cluster 4

Calculating cluster 5

Calculating cluster 6

Calculating cluster 7

Calculating cluster 8

Calculating cluster 9

Calculating cluster 10

Calculating cluster 11

Calculating cluster 12

Calculating cluster 13

Calculating cluster 14

Calculating cluster 15

Calculating cluster 16
```

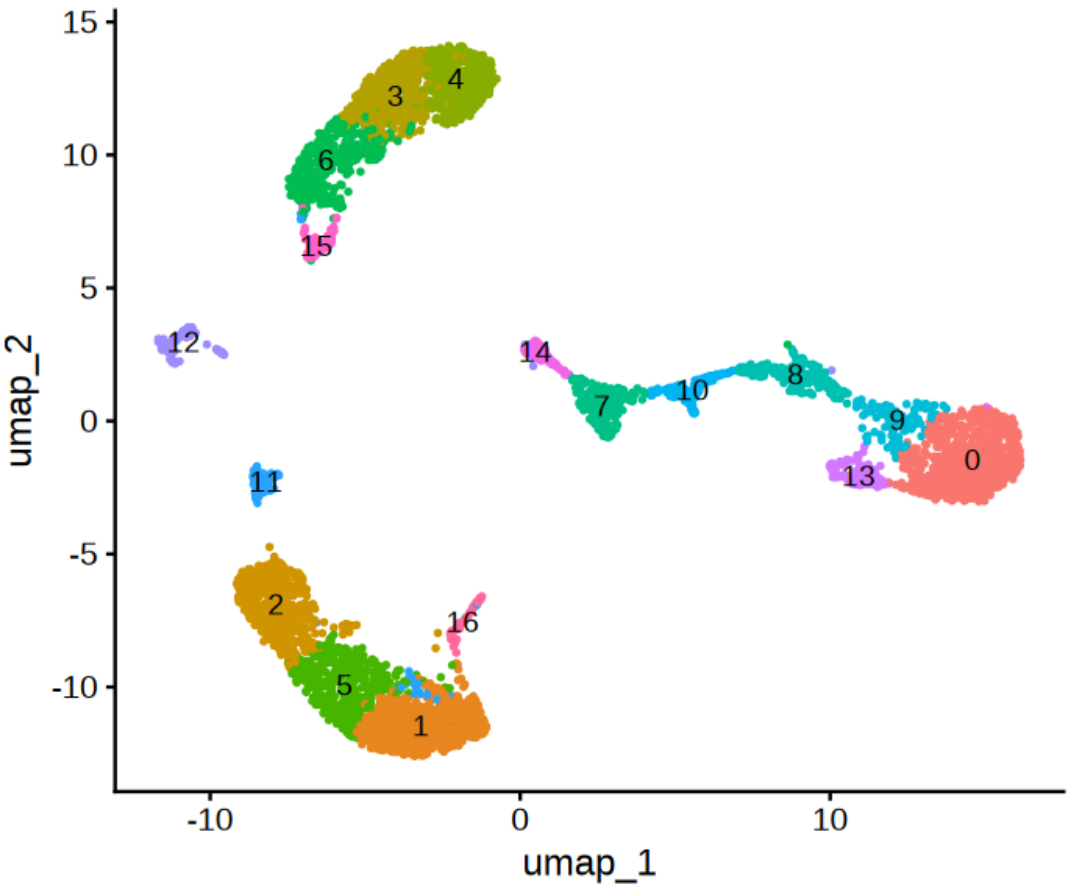
A data.frame: 6 × 7

	p_val <dbl>	avg_log2FC <dbl>	pct.1 <dbl>	pct.2 <dbl>	p_val_adj <dbl>	cluster <fct>	gene <chr>
Ltbp1	0	5.352033	0.979	0.047	0	0	Ltbp1
Ednrb	0	4.441407	0.996	0.070	0	0	Ednrb
Cd5l	0	4.209185	0.986	0.067	0	0	Cd5l
Selp	0	5.070427	0.982	0.064	0	0	Selp
Alox15	0	4.716624	0.999	0.086	0	0	Alox15
F5	0	4.792123	0.997	0.091	0	0	F5

[174]:
```
# Top 10 markers per cluster, ranked by fold change
library(dplyr)

top_markers <- markers[markers$cluster == 11,] %>%
  slice_max(order_by = avg_log2FC, n = 20)

print(top_markers$gene)
```

```
 [1] "Fcer2a"        "Gm37248"       "Ighd"          "5830444F18Rik"
 [5] "Cnn3"          "Lamb3"         "C230085N15Rik" "Mast4"
 [9] "Treml2"        "Epb41"         "Gm38380"       "Fam46c"
[13] "Bach2"         "Satb1"         "Tmem41b"       "Cbx7"
[17] "Zfp318"        "Tmem108"       "Tdrp"          "Tnik"
```

[175]:
```
options(repr.plot.width=6, repr.plot.height=5)


DimPlot(objGenes_SoloTE, reduction = "umap",label = TRUE) + NoLegend()
```



[3]:
```
#saveRDS(objGenes_SoloTE, file=paste0("data_", dataset_id, "/objGenes_SoloTE.
↪RDS"))

objGenes_SoloTE <- readRDS(file=paste0("data_", dataset_id, "/objGenes_SoloTE.
↪RDS"))
```

[14]:
```
write.table(objGenes_SoloTE$celltype, quote=FALSE, col.names = FALSE, sep =␣
↪"\t",
    file= paste0("data_",dataset_id,"/celltype_annotation.tsv"))
```

5 TEs

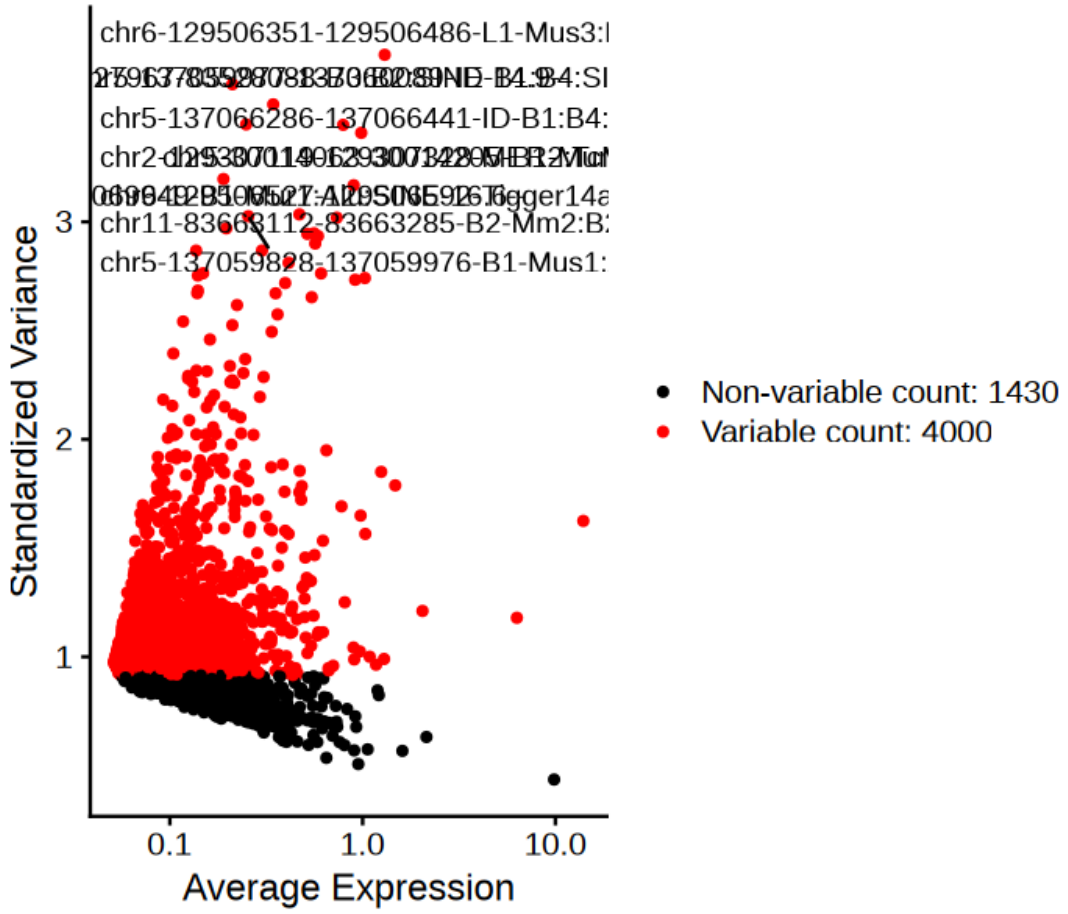
[177]:
```
### Normalize

objTE_SoloTE <- NormalizeData(objTE_SoloTE, normalization.method =␣
↪"LogNormalize", scale.factor = 10000)
```

```
objTE_SoloTE <- FindVariableFeatures(objTE_SoloTE, selection.method = "vst",␣
 ↪nfeatures = 4000)

# Identify the 10 most highly variable genes
top10 <- head(VariableFeatures(objTE_SoloTE), 10)

# plot variable features with and without labels
plot1 <- VariableFeaturePlot(objTE_SoloTE)
plot2 <- LabelPoints(plot = plot1, points = top10, repel = TRUE)
plot2
```

```
Normalizing layer: counts
```

```
Finding variable features for layer counts
```

```
When using repel, set xnudge and ynudge to 0 for optimal results
```



[178]:
```
gc()
all.genes <- rownames(objTE_SoloTE)
objTE_SoloTE <- ScaleData(objTE_SoloTE) # on hvgs

grep("MERVL", all.genes, value = T)[1:50]
```

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	23996926	1281.6	60565990	3234.6	60565990	3234.6
Vcells	366201763	2793.9	742181096	5662.4	742131242	5662.1

```
Centering and scaling data matrix
```

1. NA 2. NA 3. NA 4. NA 5. NA 6. NA 7. NA 8. NA 9. NA 10. NA 11. NA 12. NA 13. NA 14. NA 15. NA 16. NA 17. NA 18. NA 19. NA 20. NA 21. NA 22. NA 23. NA 24. NA 25. NA 26. NA 27. NA 28. NA 29. NA 30. NA 31. NA 32. NA 33. NA 34. NA 35. NA 36. NA 37. NA 38. NA 39. NA 40. NA 41. NA 42. NA 43. NA 44. NA 45. NA 46. NA 47. NA 48. NA 49. NA 50. NA

[179]:
```
objTE_SoloTE <- RunPCA(objTE_SoloTE, features = VariableFeatures(object =␣
↪objTE_SoloTE))


DimPlot(objTE_SoloTE, reduction = "pca") + NoLegend()

ElbowPlot(objTE_SoloTE)
```

```
PC_ 1
Positive:  chr11-6007429-6008777-RLTR4-MM-int:ERV1:LTR-2.9-+,
chr11-46382684-46382828-B1-Mur3:Alu:SINE-19.9-+,
chr11-46357730-46357818-MER20B:hAT-Charlie:DNA-30.7-+,
chr5-124030795-124030894-MER131:-:SINE-29.2--,
chr11-46382839-46383034-B2-Mm1a:B2:SINE-2.6-+,
chr15-60129586-60130188-L1M2:L1:LINE-31.8-+,
chr15-60152860-60158979-L1Md-F3:L1:LINE-6.7--,
chr18-5669324-5671362-L1-Mus3:L1:LINE-9.9-+,
chr14-121915156-121915339-Lx9:L1:LINE-17.4-+,
chr11-46377872-46378177-L1ME4b:L1:LINE-29.1-+
           chr18-5622306-5624101-Lx9:L1:LINE-24.0-+,
chr11-46357531-46357719-B3A:B2:SINE-25.0--,
chr13-117257629-117257908-Lx8:L1:LINE-27.1--,
chr5-124020627-124020867-B4:B4:SINE-21.7--,
chr11-86812241-86815876-MMVL30-int:ERV1:LTR-10.5--,
chr13-117269333-117269537-B3:B2:SINE-27.1-+, chr11-6008754-6010616-RLTR4-MM-
int:ERV1:LTR-8.2-+, chr13-117257909-117258092-B2-Mm2:B2:SINE-9.8-+,
chr13-117265339-117265460-PB1:Alu:SINE-29.2-+,
chr15-60143942-60145042-L1-Mus3:L1:LINE-9.0--
           chr1-131639157-131639289-B1-Mur1:Alu:SINE-18.1-+,
chr13-117265166-117265310-B1F:Alu:SINE-29.6-+,
chr15-60145057-60146797-Lx3B:L1:LINE-11.6--,
```

chr11-46324378-46324507-B1-Mm:Alu:SINE-16.1--,
chr18-5668284-5668743-RMER17D2:ERVK:LTR-11.3--,
chr11-46340115-46340234-ID4:ID:SINE-26.6--,
chr15-80652604-80653130-RMER15:ERVL:LTR-21.3-+,
chr11-52252382-52252498-PB1D9:Alu:SINE-24.8--,
chr3-35814466-35818755-L1-Mus3:L1:LINE-9.5-+,
chr11-46337737-46337950-B4:B4:SINE-29.2--
Negative: chr13-55185025-55188682-MMVL30-int:ERV1:LTR-2.9-+,
chr2-129307119-129307328-MER2:TcMar-Tigger:DNA-25.9-+,
chr6-129506351-129506486-L1-Mus3:L1:LINE-10.5--,
chr5-30014063-30014205-B1-Mur2:Alu:SINE-16.1-+,
chr6-129506527-129506592-Tigger14a:TcMar-Tigger:DNA-21.5--,
chr6-129503896-129504438-RLTR11D:ERVK:LTR-27.4-+,
chr2-23010332-23013175-MMVL30-int:ERV1:LTR-9.3--,
chr14-7938385-7938472-MIRc:MIR:SINE-25.3--,
chr7-16258634-16258742-ID-B1:B4:SINE-22.8--,
chr6-129483213-129483799-L1-Mus2:L1:LINE-7.0-+
          chr7-16258823-16258940-B1F:Alu:SINE-28.8--,
chr4-140745497-140745671-B2-Mm1a:B2:SINE-6.9--,
chr17-26504997-26505125-RLTR20C1-MM:ERVK:LTR-26.0--,
chr11-17006413-17006621-L1Md-T:L1:LINE-6.7--,
chr17-57465791-57472388-Lx:L1:LINE-15.7-+,
chr11-17006675-17006808-RSINE1:B4:SINE-31.6-+,
chr11-110100048-110100367-MTD:ERVL-MaLR:LTR-28.6--,
chr7-126624069-126624122-RSINE1:B4:SINE-29.9--,
chr1-58738844-58739082-B4A:B4:SINE-26.6-+,
chr7-126624123-126624714-RLTR18:ERVK:LTR-12.8-+
          chrX-9444985-9445178-B3:B2:SINE-22.7--,
chrX-9445718-9445881-RMER4A:ERVK:LTR-34.2--,
chr7-126623874-126624019-B1-Mus1:Alu:SINE-6.8--,
chr6-129483907-129484052-B1-Mm:Alu:SINE-7.5--,
chr2-90571556-90571763-B4A:B4:SINE-26.4--,
chr6-129503671-129503867-RMER12C:ERVK:LTR-22.6-+,
chr2-62640646-62640787-B1-Mus2:Alu:SINE-7.0--,
chr11-16996727-16997484-Lx8:L1:LINE-28.3-+,
chr6-129497568-129497695-MIR:MIR:SINE-28.8--,
chr2-90571368-90571555-B2-Mm2:B2:SINE-3.7--
PC_ 2
Positive: chr11-46382684-46382828-B1-Mur3:Alu:SINE-19.9-+,
chr11-46357730-46357818-MER20B:hAT-Charlie:DNA-30.7-+,
chr11-46382839-46383034-B2-Mm1a:B2:SINE-2.6-+,
chr15-60152860-60158979-L1Md-F3:L1:LINE-6.7--,
chr15-60129586-60130188-L1M2:L1:LINE-31.8-+,
chr11-46377872-46378177-L1ME4b:L1:LINE-29.1-+,
chr11-46357531-46357719-B3A:B2:SINE-25.0--,
chr15-60143942-60145042-L1-Mus3:L1:LINE-9.0--,
chr18-5622306-5624101-Lx9:L1:LINE-24.0-+,
chr11-46340115-46340234-ID4:ID:SINE-26.6--

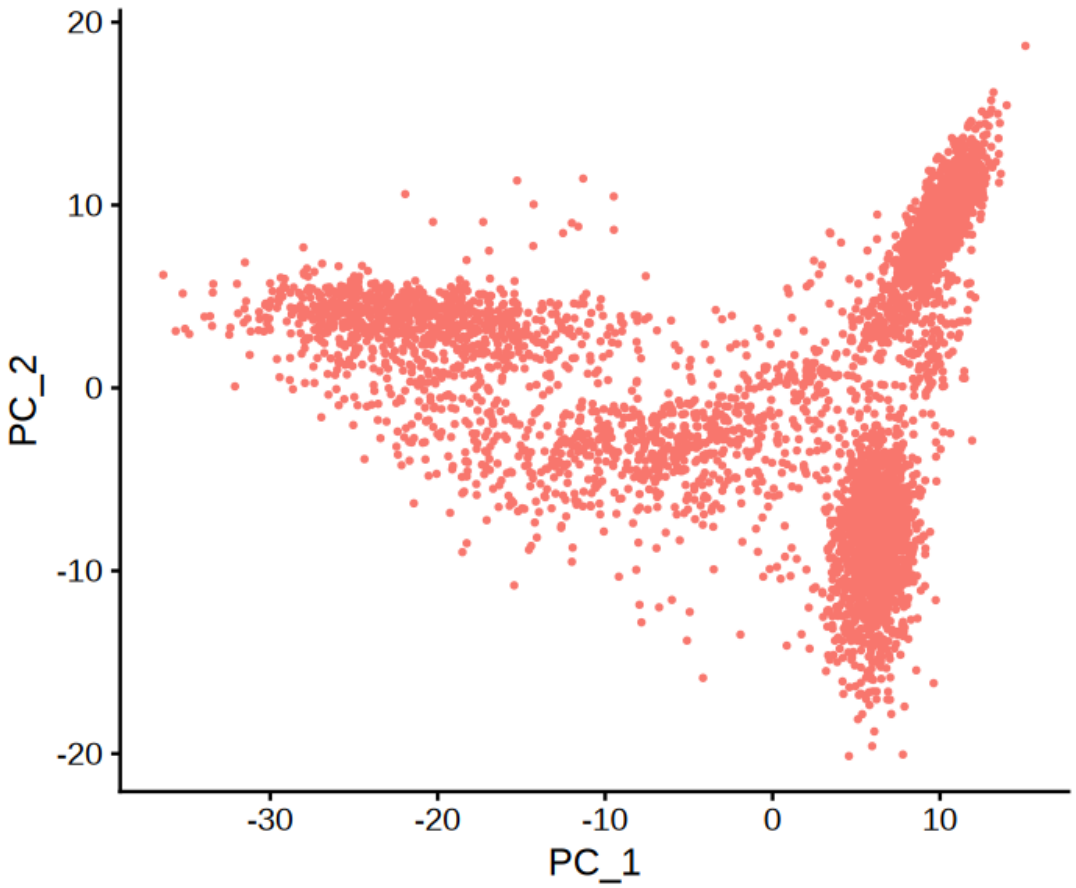
chr11-46324378-46324507-B1-Mm:Alu:SINE-16.1--,
chr15-60145057-60146797-Lx3B:L1:LINE-11.6--,
chr11-46337737-46337950-B4:B4:SINE-29.2--,
chr11-46369013-46369161-B1-Mus1:Alu:SINE-7.4--,
chr15-60123554-60123695-B1-Mus1:Alu:SINE-15.6-+,
chr11-52252382-52252498-PB1D9:Alu:SINE-24.8--,
chr11-46386144-46386223-PB1D11:Alu:SINE-16.2--,
chr11-46329103-46329235-B1-Mm:Alu:SINE-5.3--,
chr15-80597864-80598075-ID-B1:B4:SINE-28.8-+,
chr11-46382061-46382130-B1-Mur3:Alu:SINE-12.9--
chr11-46387257-46388546-RMER12C:ERVK:LTR-12.7-+,
chr6-41197707-41200061-MMVL30-int:ERV1:LTR-5.0--,
chr11-46329236-46330115-Lx9:L1:LINE-25.2-+,
chr18-5669324-5671362-L1-Mus3:L1:LINE-9.9-+,
chr5-124030795-124030894-MER131:-:SINE-29.2--,
chr15-80606960-80607164-ID-B1:B4:SINE-28.0-+,
chr11-46337115-46337528-Lx7:L1:LINE-18.5--,
chr12-73588468-73590239-L1ME2:L1:LINE-33.4-+,
chr11-46352249-46353175-L1MC4:L1:LINE-30.8--,
chr15-60143566-60143942-L1-Mus3:L1:LINE-8.0-+
Negative: chr6-128943889-128944956-Lx3B:L1:LINE-11.9--,
chr14-63416047-63416191-B1-Mm:Alu:SINE-5.5--, chr11-6007429-6008777-RLTR4-MM-
int:ERV1:LTR-2.9-+, chr11-6008754-6010616-RLTR4-MM-int:ERV1:LTR-8.2-+,
chr17-34307043-34307215-B3:B2:SINE-17.9-+, chr3-136200396-136200805-MTC:ERVL-
MaLR:LTR-18.9-+, chr13-43786860-43787000-MIR:MIR:SINE-34.8--,
chr18-60809407-60809602-B2-Mm2:B2:SINE-10.3-+,
chr3-136123600-136125156-L1VL2:L1:LINE-8.6--,
chr18-60809242-60809364-B1-Mur2:Alu:SINE-15.6--
chr3-136153968-136154216-MTD:ERVL-MaLR:LTR-24.6-+,
chr3-136194724-136194846-RSINE1:B4:SINE-31.7-+,
chr16-19161859-19166842-L1Md-F2:L1:LINE-8.0-+,
chr17-34316280-34316623-RLTR9A:ERVK:LTR-15.1--,
chr11-44700755-44700968-B3:B2:SINE-28.7-+,
chr8-81551423-81553035-L1-Mm:L1:LINE-16.9-+,
chr16-19106994-19107455-MYSERV6-int:ERVK:LTR-19.2-+,
chr16-19127235-19128081-Lx2A:L1:LINE-14.5-+,
chr14-63415279-63415541-L1MD3:L1:LINE-32.6--,
chr6-128960770-128961205-RMER17C2:ERVK:LTR-19.8-+
chr6-128964186-128966915-MYSERV16-I-int:ERVK:LTR-22.3-+,
chr18-60805991-60806100-PB1D10:Alu:SINE-20.0-+,
chr16-19203168-19203357-MMERVK10C-int:ERVK:LTR-23.7-+,
chr11-44742148-44742253-Lx8:L1:LINE-28.2-+,
chr8-83740595-83740784-B3:B2:SINE-20.0--,
chr3-136141562-136141881-L1-Mus4:L1:LINE-9.1--,
chr9-44519954-44520157-B3:B2:SINE-18.6--,
chr17-34996674-34997136-RLTR11A:ERVK:LTR-17.0--,
chr8-83740804-83740906-PB1D10:Alu:SINE-18.3--,
chr16-19293015-19297301-L1VL2:L1:LINE-11.9-+

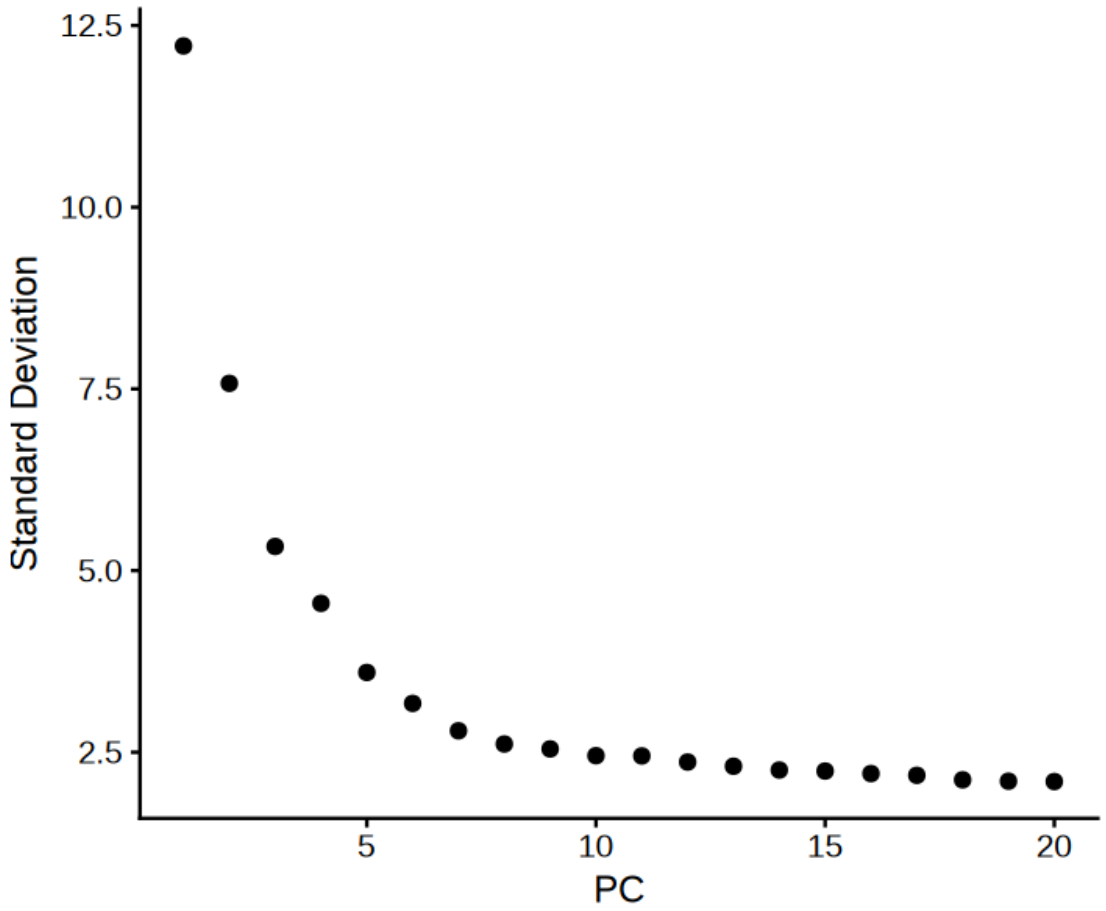
PC_ 3
Positive: chr13-44533380-44533510-B1F:Alu:SINE-25.9-+,
chr11-55110299-55110499-Charlie8:hAT-Charlie:DNA-26.5--,
chr6-134144725-134144923-ORR1F:ERVL-MaLR:LTR-30.4--,
chr9-69454697-69454778-B1F1:Alu:SINE-29.3-+,
chr15-53240495-53240619-MLT1A0:ERVL-MaLR:LTR-24.4--,
chr6-134124057-134124218-B3A:B2:SINE-36.3-+,
chr1-84139965-84141075-L1-Mur3:L1:LINE-15.2--,
chr1-84096536-84096717-B3A:B2:SINE-25.8--,
chr15-53278244-53278354-B1-Mus1:Alu:SINE-22.5-+,
chr1-84147801-84147945-MTEa:ERVL-MaLR:LTR-31.1-+
          chr15-53278251-53278369-B4:B4:SINE-30.2-+,
chr2-172400473-172400614-B1-Mur1:Alu:SINE-23.4--,
chr15-53278087-53278212-B1-Mus1:Alu:SINE-9.5--,
chr1-84149414-84150174-L1-Mus3:L1:LINE-7.7--,
chr15-53278224-53278243-MT2B:ERVL:LTR-22.3-+,
chr15-53315134-53315280-B1-Mus1:Alu:SINE-5.4--,
chr1-84135068-84135491-Lx:L1:LINE-15.9--, chr1-84101929-84102293-ORR1E:ERVL-
MaLR:LTR-30.1--, chr15-53238938-53239087-B1-Mur4:Alu:SINE-13.7--,
chr15-53225917-53226025-RSINE1:B4:SINE-28.7-+
          chr6-134036372-134036558-RMER17C:ERVK:LTR-14.8--,
chr11-83528234-83528332-B3:B2:SINE-21.6--,
chr11-83527967-83528088-B3:B2:SINE-14.9--,
chr15-53263100-53263292-B2-Mm1a:B2:SINE-3.9--,
chr15-53337905-53338051-B1-Mus1:Alu:SINE-6.8-+,
chr8-27148849-27149672-Lx7:L1:LINE-22.7-+,
chr18-39154067-39155339-L1MD:L1:LINE-28.4-+,
chr1-91079507-91079776-B4:B4:SINE-23.6-+,
chr11-6509583-6509728-B1-Mus1:Alu:SINE-6.2--,
chr13-44773401-44773552-B1-Mur4:Alu:SINE-12.2--
Negative: chr17-39842998-39843163-FordPrefect:hAT-Tip100:DNA-17.8-+,
chr17-39842889-39842997-MTC-int:ERVL-MaLR:LTR-28.4-+,
chr2-23010332-23013175-MMVL30-int:ERV1:LTR-9.3--,
chr6-129483213-129483799-L1-Mus2:L1:LINE-7.0-+,
chr14-7938385-7938472-MIRc:MIR:SINE-25.3--,
chr6-129506527-129506592-Tigger14a:TcMar-Tigger:DNA-21.5--,
chr6-129506351-129506486-L1-Mus3:L1:LINE-10.5--,
chr6-129483907-129484052-B1-Mm:Alu:SINE-7.5--,
chr4-140745497-140745671-B2-Mm1a:B2:SINE-6.9--,
chr6-129503896-129504438-RLTR11D:ERVK:LTR-27.4-+
          chr5-137059977-137060089-ID-B1:B4:SINE-25.4--,
chr6-129482260-129483212-L1-Mus3:L1:LINE-6.3--,
chr5-137059828-137059976-B1-Mus1:Alu:SINE-6.1--,
chr17-75347701-75348291-L1MA5:L1:LINE-25.7-+,
chr6-129503671-129503867-RMER12C:ERVK:LTR-22.6-+,
chr1-107518856-107519105-L1Md-F3:L1:LINE-4.1-+,
chr17-75300244-75300452-B3:B2:SINE-26.0-+,
chr9-64953583-64953681-L2a:L2:LINE-29.9-+,

chr2-33043760-33043905-B1-Mur4:Alu:SINE-11.6-+,
chr5-137066286-137066441-ID-B1:B4:SINE-30.8--
chr1-107518459-107518855-L1Md-F3:L1:LINE-5.0--,
chr17-75318159-75318591-L2a:L2:LINE-30.9--,
chr6-66128491-66133691-L1-Mus4:L1:LINE-11.7--,
chr5-137066435-137066532-PB1D10:Alu:SINE-27.1--,
chr6-129497568-129497695-MIR:MIR:SINE-28.8--,
chr1-107525539-107525653-ORR1F:ERVL-MaLR:LTR-22.1--,
chr17-75270824-75271017-MER58A:hAT-Charlie:DNA-29.3-+,
chr5-137069064-137069142-ID4-:ID:SINE-21.8-+,
chr9-64939656-64939736-RSINE1:B4:SINE-20.5--,
chr5-137069811-137069949-B1-Mur1:Alu:SINE-16.6--
PC_ 4
Positive: chr6-131246837-131246987-RSINE1:B4:SINE-28.9--,
chr7-28442082-28442384-MTC:ERVL-MaLR:LTR-18.3--,
chr1-152827864-152828009-B1-Mus2:Alu:SINE-2.0-+,
chr2-129596319-129596467-B1-Mus1:Alu:SINE-4.1-+,
chr15-82996722-82997031-Lx5b:L1:LINE-16.1--,
chr13-41390568-41390682-Tigger5:TcMar-Tigger:DNA-33.9-+,
chr7-24473033-24473238-B3:B2:SINE-18.2-+,
chr15-82993465-82993631-RSINE1:B4:SINE-30.4--,
chr6-131246597-131246736-B1-Mur3:Alu:SINE-15.9--,
chr11-17006675-17006808-RSINE1:B4:SINE-31.6-+
chr14-61657365-61657523-B4:B4:SINE-27.5--,
chr11-17006413-17006621-L1Md-T:L1:LINE-6.7--,
chr1-152818763-152818910-B1-Mus2:Alu:SINE-5.5-+,
chr9-69454697-69454778-B1F1:Alu:SINE-29.3-+,
chr7-97110943-97111020-ID4-:ID:SINE-30.8-+,
chr1-134429910-134429982-ID-B1:B4:SINE-29.8--,
chr18-36725096-36725203-RSINE1:B4:SINE-21.5--,
chr14-25600518-25600636-MIRc:MIR:SINE-27.1-+,
chr3-93522340-93522481-B1-Mus2:Alu:SINE-1.4-+,
chr7-97140154-97140564-L1MEd:L1:LINE-31.5--
chr1-152828705-152828874-Tigger7:TcMar-Tigger:DNA-23.9-+,
chr7-97133371-97135066-L1-Mur1:L1:LINE-6.5-+,
chr3-9000938-9001041-PB1D10:Alu:SINE-28.9-+,
chr19-16151217-16151495-ORR1A1:ERVL-MaLR:LTR-6.1--,
chr5-124464584-124464701-MIR:MIR:SINE-29.9--,
chr6-134144725-134144923-ORR1F:ERVL-MaLR:LTR-30.4--,
chr7-80781039-80781222-B2-Mm2:B2:SINE-7.2--,
chr6-123289427-123289524-MIR3:MIR:SINE-29.6--,
chr19-61224270-61224396-B1-Mur3:Alu:SINE-11.2-+,
chr3-8946012-8946068-ID4:ID:SINE-22.8-+
Negative: chr18-5669324-5671362-L1-Mus3:L1:LINE-9.9-+,
chr18-5622306-5624101-Lx9:L1:LINE-24.0-+,
chr1-80684590-80687755-L1-Mus1:L1:LINE-6.8--,
chr10-67206299-67206798-Lx9:L1:LINE-25.4--,
chr18-5668284-5668743-RMER17D2:ERVK:LTR-11.3--,

chr5-124030795-124030894-MER131:-:SINE-29.2--,
chr3-35814466-35818755-L1-Mus3:L1:LINE-9.5-+,
chr13-117257629-117257908-Lx8:L1:LINE-27.1--,
chr9-64902043-64902252-L1MEf:L1:LINE-31.2-+,
chr7-114074601-114074786-B3:B2:SINE-31.3--
chr13-117257909-117258092-B2-Mm2:B2:SINE-9.8-+,
chr11-46382839-46383034-B2-Mm1a:B2:SINE-2.6-+,
chr7-114074626-114074789-RSINE1:B4:SINE-27.9--,
chr15-60152860-60158979-L1Md-F3:L1:LINE-6.7--,
chr18-5668744-5668891-B1-Mm:Alu:SINE-8.2-+,
chr11-46382684-46382828-B1-Mur3:Alu:SINE-19.9-+,
chr11-46357730-46357818-MER20B:hAT-Charlie:DNA-30.7-+,
chr11-34683202-34683386-RLTR44-int:ERVK:LTR-20.6-+,
chr18-56496243-56496446-B3:B2:SINE-19.1-+,
chr13-117265339-117265460-PB1:Alu:SINE-29.2-+
chr11-46337115-46337528-Lx7:L1:LINE-18.5--,
chr10-67191849-67191946-PB1D7:Alu:SINE-30.9--,
chr18-5671402-5674317-L1-Mus3:L1:LINE-9.6-+,
chr3-101283526-101283713-B2-Mm2:B2:SINE-16.9--,
chr11-46357531-46357719-B3A:B2:SINE-25.0--,
chr14-121915156-121915339-Lx9:L1:LINE-17.4-+,
chr5-124020627-124020867-B4:B4:SINE-21.7--,
chr5-124029080-124029272-B2-Mm2:B2:SINE-18.5--,
chr7-114074822-114075015-L1MB8:L1:LINE-24.7--,
chr18-56496453-56496644-B2-Mm2:B2:SINE-8.5-+
PC_ 5
Positive: chr7-97131031-97133325-L1-Mur1:L1:LINE-7.7-+,
chr14-61676214-61676401-B2-Mm2:B2:SINE-9.7--,
chr7-97133371-97135066-L1-Mur1:L1:LINE-6.5-+,
chr7-97140154-97140564-L1MEd:L1:LINE-31.5--,
chr14-61658154-61658251-L3:CR1:LINE-30.6--,
chr1-152827864-152828009-B1-Mus2:Alu:SINE-2.0-+,
chr13-41415200-41415659-L1MA5:L1:LINE-27.6--,
chr7-97110943-97111020-ID4-:ID:SINE-30.8-+,
chr7-97083718-97083850-ID-B1:B4:SINE-27.3-+,
chr1-152828705-152828874-Tigger7:TcMar-Tigger:DNA-23.9-+
chr8-83740595-83740784-B3:B2:SINE-20.0--,
chr7-97083522-97083655-RSINE1:B4:SINE-24.6-+,
chr14-61657365-61657523-B4:B4:SINE-27.5--,
chr8-83740804-83740906-PB1D10:Alu:SINE-18.3--,
chr14-61670397-61670583-B4A:B4:SINE-20.0--,
chr19-5830694-5831063-L1MB7:L1:LINE-29.3-+,
chr14-61659187-61659321-B1-Mus2:Alu:SINE-7.5--,
chr7-117516507-117516627-PB1D11:Alu:SINE-28.9-+,
chr1-152813371-152813479-B4A:B4:SINE-22.6--,
chr7-117516640-117516718-ID4-:ID:SINE-21.4-+
chr8-117281405-117281597-B2-Mm1a:B2:SINE-4.2-+,
chr7-97083755-97083882-B1F:Alu:SINE-30.2-+,

chr7-24468677-24468914-B4A:B4:SINE-25.1--,
chr3-8996552-8997789-L1-Mus1:L1:LINE-5.3--,
chr7-97155550-97155966-L1-Mus3:L1:LINE-7.8-+,
chr17-48233906-48234115-B3:B2:SINE-26.8--,
chr8-117296379-117296749-RMER4B:ERVK:LTR-14.9-+,
chr14-61662221-61662726-RLTR16:ERVK:LTR-18.5-+,
chr13-41455889-41455979-L2a:L2:LINE-31.9-+,
chr3-9000938-9001041-PB1D10:Alu:SINE-28.9-+
Negative: chr17-39842998-39843163-FordPrefect:hAT-Tip100:DNA-17.8-+,
chr17-39842889-39842997-MTC-int:ERVL-MaLR:LTR-28.4-+,
chr11-83528234-83528332-B3:B2:SINE-21.6--,
chr11-83527967-83528088-B3:B2:SINE-14.9--,
chr17-34316280-34316623-RLTR9A:ERVK:LTR-15.1--,
chr17-34307043-34307215-B3:B2:SINE-17.9-+,
chr11-83663112-83663285-B2-Mm2:B2:SINE-9.2-+,
chr11-83664807-83665072-L1-Mus1:L1:LINE-9.1-+,
chr17-34308690-34308852-Lx9:L1:LINE-14.4-+,
chr2-62637824-62637970-B4A:B4:SINE-32.9-+
chr4-32159388-32160005-Lx2B:L1:LINE-16.8--,
chr11-86812241-86815876-MMVL30-int:ERV1:LTR-10.5--,
chr2-62632553-62632724-Lx:L1:LINE-6.6--,
chr2-62640646-62640787-B1-Mus2:Alu:SINE-7.0--, chr17-34312948-34313293-MTC:ERVL-
MaLR:LTR-16.1--, chr4-135249055-135249199-B1-Mur4:Alu:SINE-10.4--,
chr18-60809242-60809364-B1-Mur2:Alu:SINE-15.6--,
chr18-60809407-60809602-B2-Mm2:B2:SINE-10.3-+,
chr2-61610466-61610807-Lx5:L1:LINE-18.0-+,
chr4-135248861-135249051-B2-Mm2:B2:SINE-7.5--
chr2-62639185-62639201-Lx3C:L1:LINE-14.5--,
chr17-34282389-34282507-B1F2:Alu:SINE-30.2--,
chr15-53240495-53240619-MLT1A0:ERVL-MaLR:LTR-24.4--,
chr4-135232235-135232382-B1-Mus1:Alu:SINE-8.1--,
chr17-34308856-34309105-Lx7:L1:LINE-25.8-+, chr2-129307119-129307328-MER2:TcMar-
Tigger:DNA-25.9-+, chr4-135271870-135271964-B1F:Alu:SINE-19.1--,
chr3-135616065-135616336-Lx7:L1:LINE-25.9--,
chr15-53278087-53278212-B1-Mus1:Alu:SINE-9.5--,
chr4-135234275-135234420-B1-Mus1:Alu:SINE-8.2--


20
10
0
-10
-20
PC_2
-30
-20
-10
0
10
PC_1

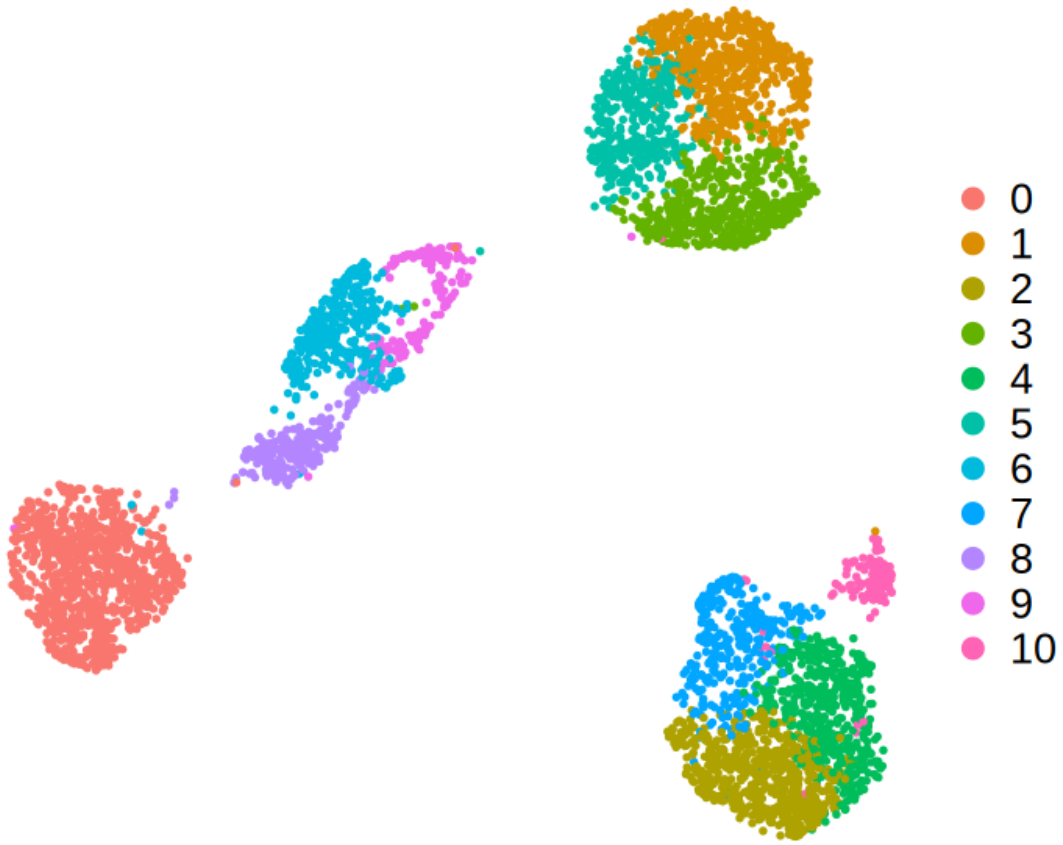


[180]:
```
objTE_SoloTE <- FindNeighbors(objTE_SoloTE, dims = 1:10, k.param = 20)
objTE_SoloTE <- FindClusters(objTE_SoloTE, resolution = 1)
objTE_SoloTE <- RunUMAP(objTE_SoloTE, dims = 1:10)
DimPlot(objTE_SoloTE, reduction = "umap") +
    theme_void() +
    theme(text=element_text(size=20))
```

```
Computing nearest neighbor graph


Computing SNN


Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
```

```
Maximum modularity in 10 random starts: 0.8211
Number of communities: 11
Elapsed time: 0 seconds

11:08:04 UMAP embedding parameters a = 0.9922 b = 1.112

11:08:04 Read 4787 rows and found 10 numeric columns

11:08:04 Using Annoy for neighbor search, n_neighbors = 30

11:08:04 Building Annoy index with metric = cosine, n_trees = 50

0%   10   20   30   40   50   60   70   80   90   100%

[----|----|----|----|----|----|----|----|----|----|

*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
```

```
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
*
|

11:08:05 Writing NN index file to temp file /tmp/RtmpNvYOrB/file1b1319797c4f5

11:08:05 Searching Annoy index using 1 thread, search_k = 3000

11:08:06 Annoy recall = 100%

11:08:10 Commencing smooth kNN distance calibration using 1 thread
 with target n_neighbors = 30

11:08:18 Initializing from normalized Laplacian + noise (using RSpectra)

11:08:18 Commencing optimization for 500 epochs, with 193788 positive edges

11:08:18 Using rng type: pcg

11:08:27 Optimization finished
```



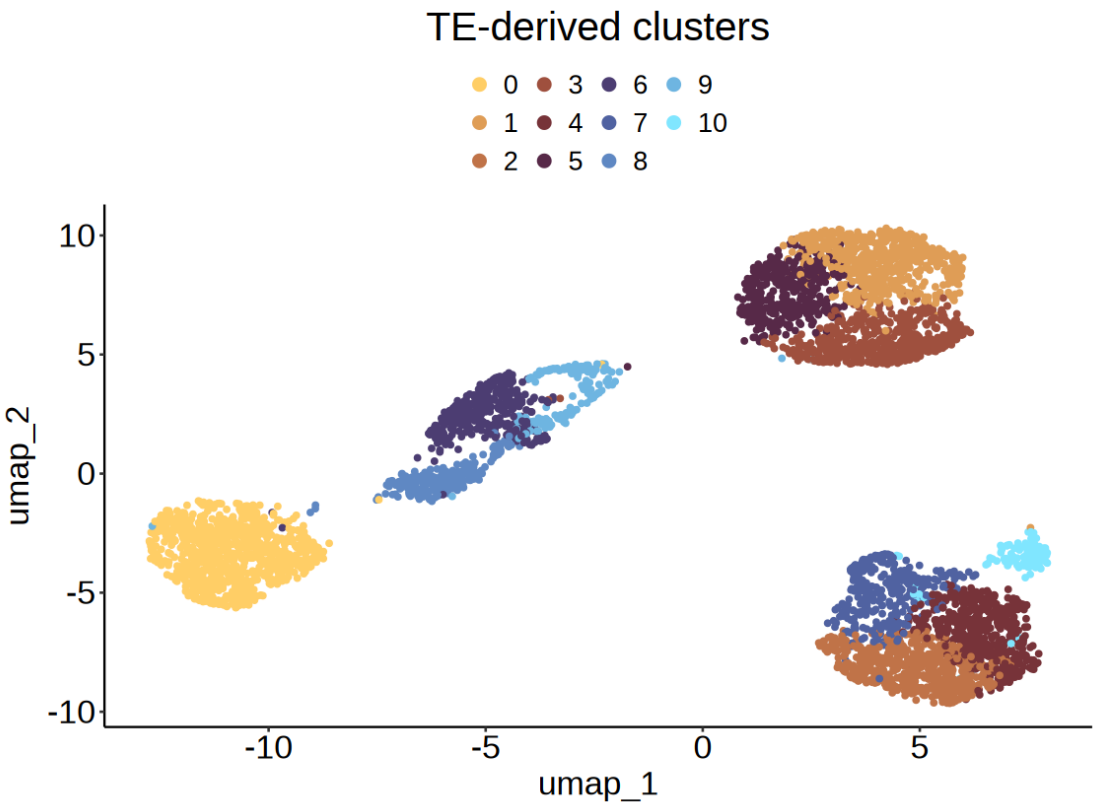
6 Checkpoint

[181]:
```
saveRDS(objTE_SoloTE, paste0("data_", dataset_id, "/soloTE_", dataset_id,␣
↪"_seuratObj.RDS"))
#objTE_SoloTE <- readRDS(paste0("data_", dataset_id, "/soloTE_", dataset_id,␣
↪"_seuratObj.RDS"))
```

[182]:
```
options(repr.plot.width=9.5, repr.plot.height=7)
library(ggpubr)
library(scico)

DimPlot(objTE_SoloTE, reduction = "umap",
        shuffle=T, pt.size = 1) +
        ggtitle("TE-derived clusters") +
  scale_color_manual(values=␣
↪scico(length(unique(objTE_SoloTE$seurat_clusters)), palette = 'managua')) +
  theme_pubr() +
```

```
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_",dataset_id,"/umap_TEs_locus_clusters_SoloTE_scico.
  ↪pdf"), device = "pdf", width=9.5, height=7)
```

```
Warning message:
"The `size` argument of `element_line()` is deprecated as of ggplot2
3.4.0.
  Please use the `linewidth` argument instead.
  The deprecated feature was likely used in the ggpubr
package.
   Please report the issue at
<https://github.com/kassambara/ggpubr/issues>."
Warning message:
"The `size` argument of `element_rect()` is deprecated as of ggplot2
3.4.0.
  Please use the `linewidth` argument instead.
  The deprecated feature was likely used in the ggpubr
package.
   Please report the issue at
<https://github.com/kassambara/ggpubr/issues>."
```



[36]:
```
scico(30, palette = 'managua')
#>  [1] "#190C64" "#1C176B" "#202272" "#212B79" "#243580" "#263D86" "#29478B"
#>  [8] "#2C5091" "#2F5996" "#33619A" "#37699D" "#3D71A0" "#4479A1" "#4D81A2"
#> [15] "#5688A4" "#608EA2" "#6B94A1" "#77999F" "#839E9C" "#90A198" "#9BA495"
#> [22] "#A9A895" "#B7AD96" "#C7B59C" "#D7BEA6" "#E5C9B3" "#F0D4C3" "#F7DFD3"
#> [29] "#FCE9E3" "#FEF2F2"
```

1. '#FFCE66' 2. '#F4BD61' 3. '#E9AC5B' 4. '#DE9C56' 5. '#D38D50' 6. '#C97F4C' 7. '#BE7148' 8. '#B36444' 9. '#A7583F' 10. '#9B4C3D' 11. '#8E423A' 12. '#7F3839' 13. '#712F39' 14. '#65293D' 15. '#5B2843' 16. '#54294D' 17. '#4F305C' 18. '#4B396B' 19. '#4C447D' 20. '#4C518E' 21. '#4F5D9C' 22. '#536BAA' 23. '#5878B6' 24. '#5D86C1' 25. '#6395CC' 26. '#68A4D5' 27. '#6EB4E0' 28. '#74C4EA' 29. '#7AD5F4' 30. '#80E6FF'

[ ]:
```

```

7 Compare cluters

[183]:
```
clustersResList <- list()
identical(Cells(objGenes_SoloTE), Cells(objTE_SoloTE)) # TRUE, same cells in␣
 ↪same order

for(res in seq(0.5, 2, by=0.1)){
    print(res)

    clusters_genes <- FindClusters(objGenes_SoloTE, resolution = res)
    clusters_TEs <- FindClusters(objTE_SoloTE, resolution = res)

    clustersResList[[as.character(res)]] <-␣
 ↪cbind(clusters_genes$seurat_clusters, clusters_TEs$seurat_clusters)
    #
}
```

TRUE

```
[1] 0.5
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.9100
Number of communities: 13
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
```

```
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8858
Number of communities: 8
Elapsed time: 0 seconds
[1] 0.6
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8992
Number of communities: 14
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8695
Number of communities: 9
Elapsed time: 0 seconds
[1] 0.7
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8881
Number of communities: 14
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8560
Number of communities: 11
Elapsed time: 0 seconds
[1] 0.8
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
```

```
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8787
Number of communities: 17
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8447
Number of communities: 11
Elapsed time: 0 seconds
[1] 0.9
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8701
Number of communities: 17
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8327
Number of communities: 11
Elapsed time: 0 seconds
[1] 1
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8618
Number of communities: 17
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863
```

```
Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8211
Number of communities: 11
Elapsed time: 0 seconds
[1] 1.1
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8533
Number of communities: 17
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8118
Number of communities: 13
Elapsed time: 0 seconds
[1] 1.2
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8450
Number of communities: 17
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8013
Number of communities: 14
Elapsed time: 0 seconds
[1] 1.3
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416
```

```
Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8365
Number of communities: 18
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7917
Number of communities: 15
Elapsed time: 0 seconds
[1] 1.4
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8295
Number of communities: 18
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7828
Number of communities: 15
Elapsed time: 0 seconds
[1] 1.5
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8225
Number of communities: 21
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863
```

```
Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7777
Number of communities: 19
Elapsed time: 0 seconds
[1] 1.6
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8169
Number of communities: 21
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7705
Number of communities: 19
Elapsed time: 0 seconds
[1] 1.7
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8105
Number of communities: 22
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7642
Number of communities: 21
Elapsed time: 0 seconds
[1] 1.8
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416
```

```
Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.8052
Number of communities: 23
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7590
Number of communities: 21
Elapsed time: 0 seconds
[1] 1.9
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7985
Number of communities: 23
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7537
Number of communities: 21
Elapsed time: 0 seconds
[1] 2
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 161416

Running Louvain algorithm…
Maximum modularity in 10 random starts: 0.7931
Number of communities: 23
Elapsed time: 0 seconds
Modularity Optimizer version 1.3.0 by Ludo Waltman and Nees Jan van Eck

Number of nodes: 4787
Number of edges: 166863

Running Louvain algorithm…
```

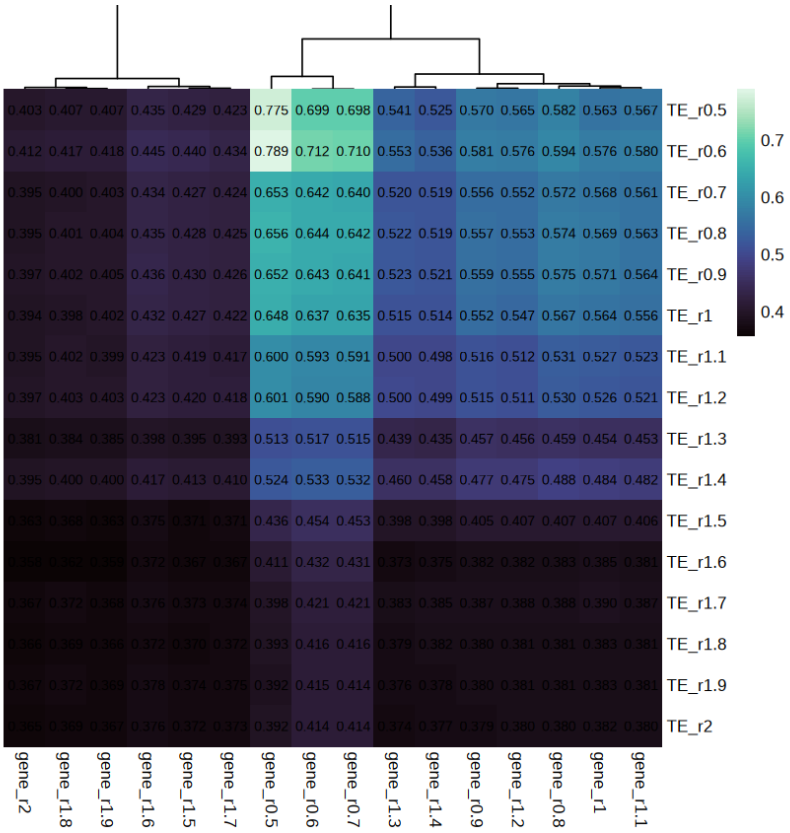
```
Maximum modularity in 10 random starts: 0.7484
Number of communities: 21
Elapsed time: 0 seconds
```

[184]:
```r
library(mclust)
library(viridis)

resolutions <- seq(0.5, 2.0, by = 0.1)

ari_matrix <- matrix(data=NA, nrow=length(resolutions),ncol=length(resolutions))
colnames(ari_matrix) <- paste0("gene_r",resolutions)
rownames(ari_matrix) <- paste0("TE_r",resolutions)

for(r_gene in resolutions){
    for(r_TE in resolutions){
        ari <- adjustedRandIndex(clustersResList[[as.character(r_gene)]][,1],
                    clustersResList[[as.character(r_TE)]][,2])
        ari_matrix[paste0("TE_r",r_TE),paste0("gene_r",r_gene)] <- ari
    }
}
ari_matrix

pheatmap(ari_matrix, display_numbers = T, color = mako(100, alpha = 1, begin = 0, end = 1, direction = 1)[],
         border_color = NA, number_format = "%.3f", number_color="black",
         cluster_rows = FALSE, cluster_columns = FALSE,
         cellwidth = 25, cellheight = 25)


# ari_values <- sapply(resolutions, function(r) {
#     adjustedRandIndex(clustersResList[[as.character(r)]][,1],
#                       clustersResList[[as.character(r)]][,2])
# })

# plot(resolutions, ari_values, type = "b",
#      xlab = "Resolution", ylab = "ARI",
#      main = "Gene vs Transposon Clustering Similarity")
```

```
Package 'mclust' version 6.1.1
Type 'citation("mclust")' for citing this R package in publications.


Loading required package: viridisLite

Warning message:
"package ‘viridisLite’ was built under R version 4.4.3"
```

A matrix: 16 × 16 of type dbl

	gene__r0.5	gene__r0.6	gene__r0.7	gene__r0.8	gene__r0.9	gene__r1
TE__r0.5	0.7750790	0.6989213	0.6983347	0.5817911	0.5697554	0.563460
TE__r0.6	0.7885226	0.7123767	0.7103633	0.5943729	0.5811188	0.575634
TE__r0.7	0.6526793	0.6420946	0.6401397	0.5722579	0.5557949	0.568440
TE__r0.8	0.6562326	0.6440560	0.6421046	0.5737891	0.5573397	0.569058
TE__r0.9	0.6524176	0.6425461	0.6405935	0.5745733	0.5588831	0.570776
TE__r1	0.6482636	0.6368609	0.6348804	0.5671057	0.5523523	0.564361
TE__r1.1	0.5995461	0.5930339	0.5913376	0.5309853	0.5155187	0.526707
TE__r1.2	0.6009048	0.5900235	0.5883532	0.5296173	0.5147510	0.525515
TE__r1.3	0.5125682	0.5165767	0.5153782	0.4593730	0.4570273	0.453851
TE__r1.4	0.5236827	0.5331734	0.5323518	0.4878886	0.4773532	0.483962
TE__r1.5	0.4355497	0.4535911	0.4526610	0.4073503	0.4054127	0.406999
TE__r1.6	0.4107503	0.4321910	0.4313005	0.3830272	0.3818945	0.384871
TE__r1.7	0.3982946	0.4211623	0.4211490	0.3878360	0.3866106	0.390007
TE__r1.8	0.3929192	0.4156372	0.4157357	0.3809480	0.3804222	0.383350
TE__r1.9	0.3920631	0.4147882	0.4144816	0.3812182	0.3802414	0.383004
TE__r2	0.3922642	0.4144241	0.4141237	0.3804765	0.3790087	0.382240



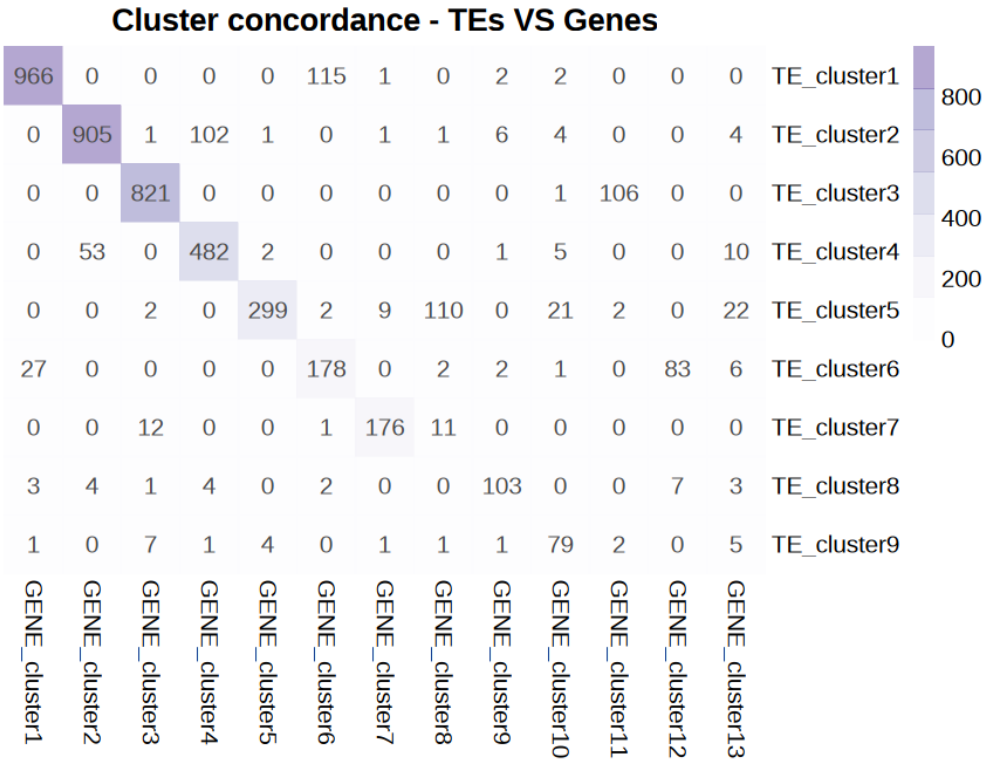
[188]:
```
TEclusters <- clustersResList[["0.6"]][,2]
table(clustersResList[["0.6"]][,2])
```

```
GENEclusters <- clustersResList[["0.5"]][,1]
table(clustersResList[["0.5"]][,1])
```

```
   1    2    3    4    5    6    7    8    9
1086 1025  928  553  467  299  200  127  102
```

```
  1   2   3   4   5   6   7   8   9  10  11  12  13
997 962 844 589 306 298 188 125 115 113 110  90  50
```

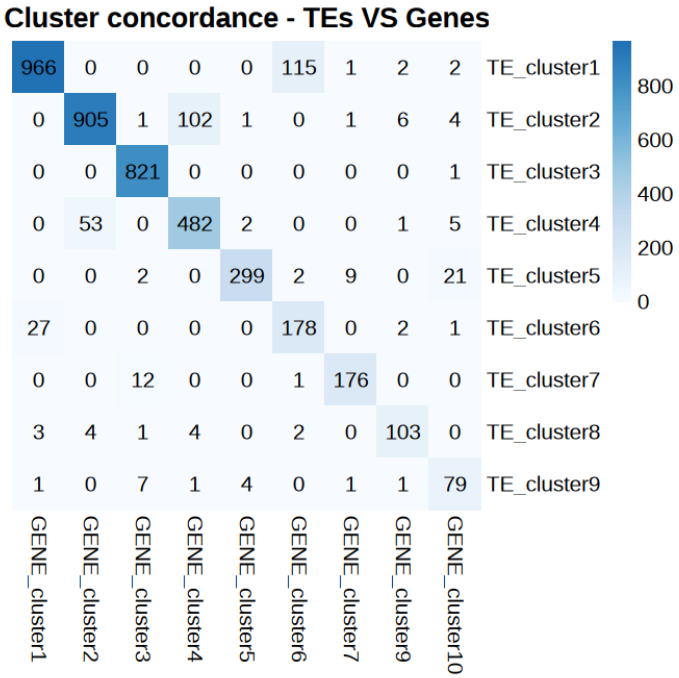
[186]:
```
options(repr.plot.width=8, repr.plot.height=7)

concordanceTable <- table(TEclusters, GENEclusters)

rownames(concordanceTable) <- paste0("TE_cluster", rownames(concordanceTable))
colnames(concordanceTable) <- paste0("GENE_cluster", colnames(concordanceTable))

pheatmap(concordanceTable, display_numbers = T,main = "Cluster concordance -␣
 ↪TEs VS Genes",
       fontsize_number = 12, fontsize = 12, annotation_names_row = TRUE,
       color = alpha(brewer.pal(9,'Purples')[1:7], 0.5),
       cluster_rows=FALSE, cluster_cols=FALSE,
        border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight␣
 ↪= 30)
```



[187]:
```r
library(clue)
library(grid)
options(repr.plot.width=9, repr.plot.height=8)

# Suppose your matrix has more rows than columns
nr <- nrow(concordanceTable)
nc <- ncol(concordanceTable)

if(nr > nc){
  # pad with zeros to make it square
  M <- cbind(concordanceTable, matrix(0, nrow = nr, ncol = nr - nc))
} else {
  M <- concordanceTable
}

perm <- solve_LSAP(M, maximum = TRUE)
```

```r
# keep only the original columns
M_reordered <- M[, perm[1:nc]]



ph <- pheatmap(M_reordered[,!is.na(colnames(M_reordered))],
        display_numbers = T, color = colorRampPalette(brewer.pal(9,'Blues')[1:
 ↪7])(100),#adjustcolor((colorRampPalette(carto_pal(name="Emrld")))(100),alpha.
 ↪f=1),
        main = "Cluster concordance - TEs VS Genes",
        cluster_rows=FALSE, cluster_cols=FALSE,
                fontsize_number = 12, fontsize = 12, annotation_names_row =␣
 ↪TRUE, number_color = "black",
         border_color = NA, number_format = "%.0f", cellwidth = 30, cellheight␣
 ↪= 30)


# write to PDF safely

pdf(paste0("figures_", dataset_id, "/
 ↪heatmap_cluster_clusters_concordance_SoloTE.pdf"), width=8, height=9)

grid::grid.newpage()
grid::grid.draw(ph$gtable)

dev.off()
```

agg__record__2014549349: 2



[189]:
```
unique(objGenes_SoloTE$celltype)
```

1. 'Monocytes' 2. 'Dendritic Cells' 3. 'T_cells' 4. 'B_cells' 5. 'Natural Killers' 6. 'Platelets' 7. 'Neutrophils' 8. 'Unknown'

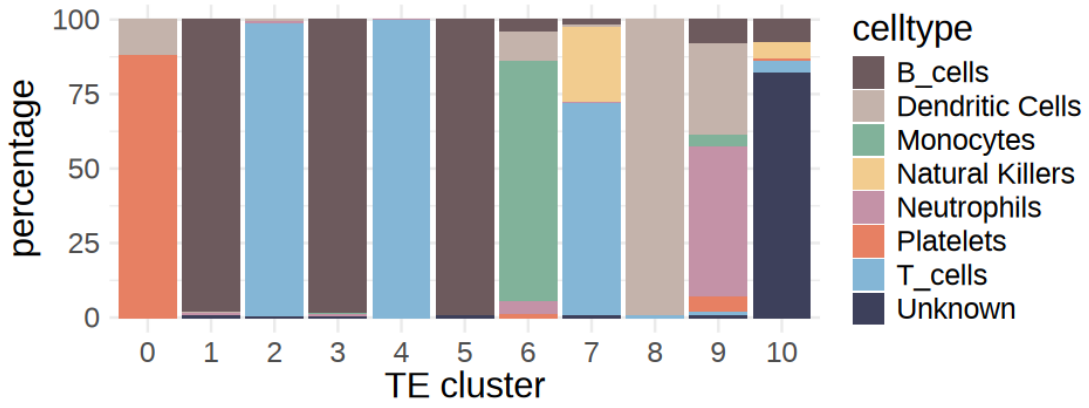
[228]:
```
PBMCcelltypeColors <- c("B_cells"="#6D5A5D",
                        "Dendritic Cells"="#C4B3AB", #"#E78063",#"#c492a7",
                        "Monocytes"="#81B29A",
                        "Platelets"="#e78063",
                        "T_cells"="#84B6D6",
                        "Natural Killers"= "#F2CC8F",
                        "Neutrophils"="#c492a7",
                        "Unknown" = "#3D405B")
```

[229]:
```
#add cell types to TE obj
identical(rownames(objTE_SoloTE@meta.data),
    rownames(objGenes_SoloTE@meta.data) ) # same cells in same order
```

```
objTE_SoloTE$celltype <- objGenes_SoloTE$celltype
```

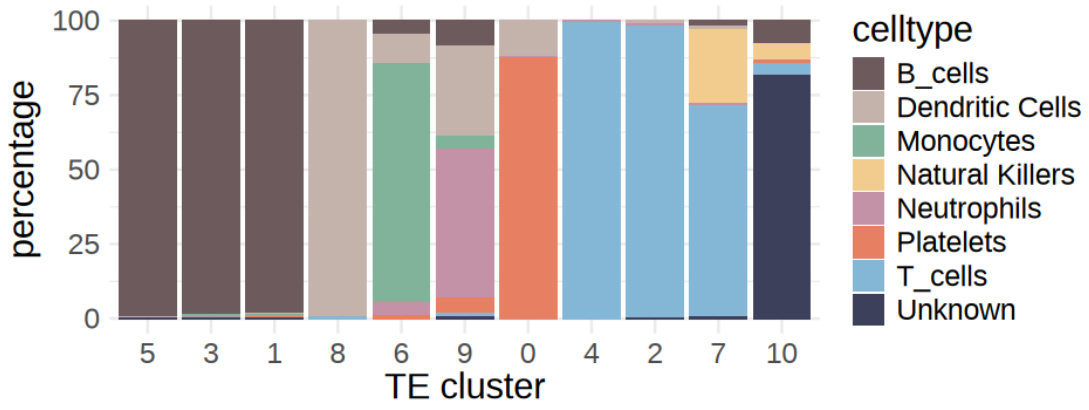
TRUE

[230]:
```
options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster","celltype","nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

df$cluster <- factor(df$cluster, levels=c(0,1:length(unique(df$cluster))))


ggplot(df, aes(x=cluster, y=percentage, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  scale_fill_manual(values=PBMCcelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))
ggsave(paste0("figures_", dataset_id,"/clusterIdentity_barplot_SoloTE.pdf"),␣
 ↪width=8, height=3)
```



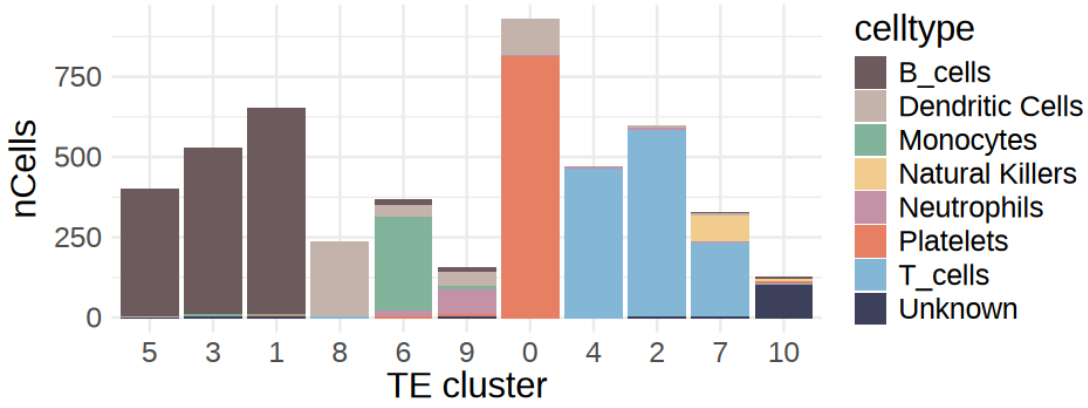
[237]:
```
options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster","celltype","nCells")
df$cluster <- as.character(df$cluster)
```

```
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

df <- df %>%
  group_by(cluster) %>%
  mutate(main = celltype[which.max(percentage)]) %>%
  ungroup() %>%
  arrange(main, desc(percentage))
df$cluster <- factor(df$cluster, levels = unique(df$cluster))


ggplot(df, aes(x=cluster, y=percentage, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  # geom_text(data = sizes,
  #          aes(x = factor(cluster), y = 105, label = clusterSize),
  #          inherit.aes = FALSE) +
  scale_fill_manual(values=PBMCcelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))
ggsave(paste0("figures_", dataset_id,"/
 ↪clusterIdentity_perc_barplot_SoloTE_noplatelet.pdf"), width=8, height=3)
```



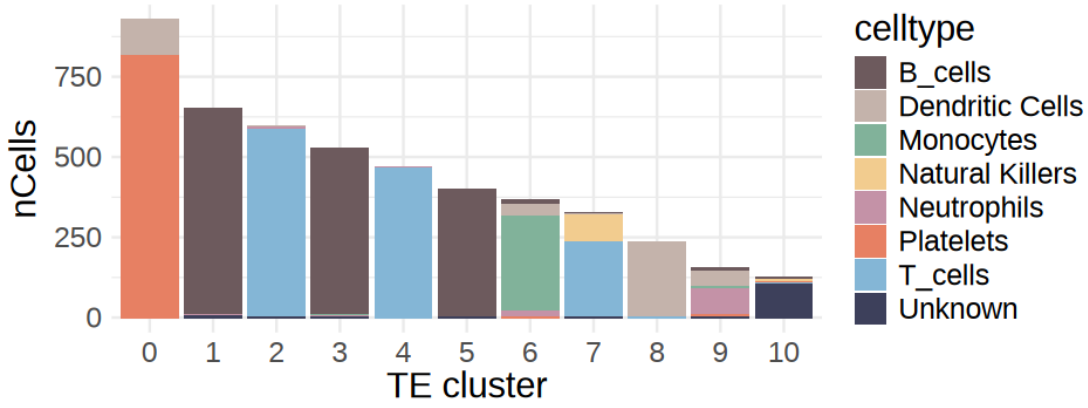
[236]:
```
options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster","celltype","nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)
```

```
df <- df %>%
  group_by(cluster) %>%
  mutate(main = celltype[which.max(percentage)]) %>%
  ungroup() %>%
  arrange(main, desc(percentage))
df$cluster <- factor(df$cluster, levels = unique(df$cluster))


ggplot(df, aes(x=cluster, y=nCells, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  # geom_text(data = sizes,
  #           aes(x = factor(cluster), y = 105, label = clusterSize),
  #           inherit.aes = FALSE) +
  scale_fill_manual(values=PBMCcelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))
ggsave(paste0("figures_", dataset_id,"/
 ↪clusterIdentity_barplot_SoloTE_noplatelet.pdf"), width=8, height=3)
```

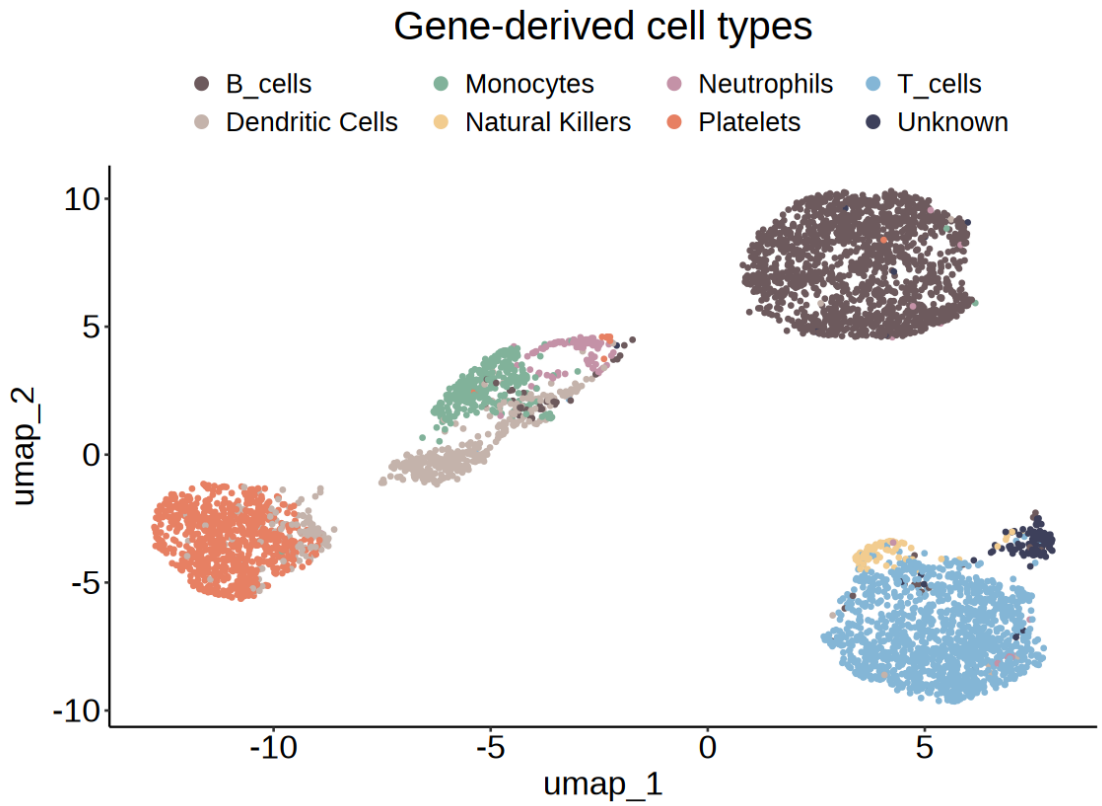


[235]:
```
options(repr.plot.width=8, repr.plot.height=3)
concordanceTable <- table(objTE_SoloTE$seurat_clusters, objTE_SoloTE$celltype)

df <- melt(concordanceTable)
df <- df[df$value!=0,]
colnames(df) <- c("cluster","celltype","nCells")
df$cluster <- as.character(df$cluster)
df$clusterSize <- table(objTE_SoloTE$seurat_clusters)[df$cluster]
df$percentage <- as.numeric(df$nCells / df$clusterSize *100)

df <- df %>%
  group_by(cluster) %>%
  #mutate(main = celltype[which.max(percentage)]) %>%
```

```
  #ungroup() %>%
  arrange(desc(clusterSize))
df$cluster <- factor(df$cluster, levels = unique(df$cluster))


ggplot(df, aes(x=cluster, y=nCells, fill=celltype)) +
  geom_col() +
  xlab("TE cluster") +
  # geom_text(data = sizes,
  #          aes(x = factor(cluster), y = 105, label = clusterSize),
  #          inherit.aes = FALSE) +
  scale_fill_manual(values=PBMCcelltypeColors)+
  theme_minimal() + theme(text=element_text(size=18))
ggsave(paste0("figures_", dataset_id,"/
 ↪clusterIdentity_barplot_SoloTE_noplatelet_orderedBySize.pdf"), width=8,␣
 ↪height=3)
```



[234]:
```
options(repr.plot.width=9.5, repr.plot.height=7)

DimPlot(objTE_SoloTE, reduction = "umap", group.by = "celltype",
        cols=PBMCcelltypeColors,
        shuffle=T, pt.size = 0.7) +
        ggtitle("Gene-derived cell types") +
  theme_pubr() +
  theme(text=element_text(size=20), plot.title = element_text(hjust=0.5))
ggsave(paste0("figures_",dataset_id,"/umap_TEs_locus_celltypes_SoloTE.pdf"),␣
 ↪device = "pdf", width=9.5, height=7)
```


Gene-derived cell types
B_cells
Monocytes
Neutrophils
T_cells
Dendritic Cells
Natural Killers
Platelets
Unknown
10
5
0
-5
-10
umap_2
-10
-5
0
5
umap_1

[ ]: